SignalTree v15

Engineering a Reactive State Engine

Architecture, semantics, falsification, performance, lifecycle, and the evidence behind the rewrite

Engineering whitepaper - architecture cut: August 25, 2026

Document status: Pre-RC engineering whitepaper. Core architecture and the public Link/error contract are substantially closed and frozen absent a new falsifier. MIGRATION-MAP-0 is complete; compatibility retirement now begins with an asyncQuery disposition, followed by loader/asyncSource/stored-persistence cleanup. Full demo coverage, performance-proof correction, final release gates, and owned-history reclamation remain open.

This paper is reconstructed from the v15 engineering record: code audits, commit history, decision documents, benchmark outputs, falsification probes, type-contract tests, GC tests, and the latest lifecycle discriminator. The separately supplied Signaltree15.pdf was explicitly excluded as a source.

Abstract

SignalTree v15 is not a cosmetic API release. It is the result of a broad rewrite and adversarial audit of how SignalTree constructs stores, represents structured state, identifies entity lifetimes, records causal meaning, publishes reactive change, retains history, and proves its own performance claims.

The work began with a simple goal - make SignalTree fast, correct, and architecturally durable - but repeatedly overturned assumptions that initially appeared settled. A materialized entity projection was proven to make all() faster, then removed because its permanent memory cost did not earn itself across the actual workload model. Weak interning of entity signals passed the full ordinary test suite and cut memory substantially, then was rejected because a forced-GC probe exposed silent stale UI. Retired entity records were first partially reclaimed, then the remaining tombstone ledger itself was falsified: stale-handle isolation survived complete zero-owner lifetime forgetting, converting an unbounded retirement cost into a bounded asymptote. An apparent memory leak across repeated store creation was ultimately shown to be a tree lifecycle contract: a write-active SignalTree remains retained until destroy() is called. An unrelated transaction rollback defect was uncovered because a lifecycle control could not be made to pass; the root cause proved that subject identity and leaf address are orthogonal dimensions of semantic state location.

Late in hardening, the same falsification discipline was applied to external synchronization. A production link() relationship was reduced to a three-method handle (retrieve, settled, dispose), proven against strong whole-relationship settlement, full-value synchronization, structural equality, echo suppression, failure recovery, and collection snapshots. The generic error observer was repaired before publication: events now carry a required runtime-local TreeId and one coherent state path, while dead source/detail taxonomy was deleted rather than hidden.

The final v15 direction is therefore defined less by one data structure than by a set of separations:

logical address is not subject lifetime identity;

subject identity is not revision identity;

physical build capability is not restoration authority;

physical truth is not causal authorship;

publication is not persistence;

external synchronization is not mutation authority;

a Link boundary exchanges complete values even when internal reactivity is granular;

diagnostic location is not tree identity;

reactive observation is not state ownership;

a whole-collection projection is not the same thing as point access disguised by a missing index;

a tree's lifetime is not the same thing as a subject's lifetime;

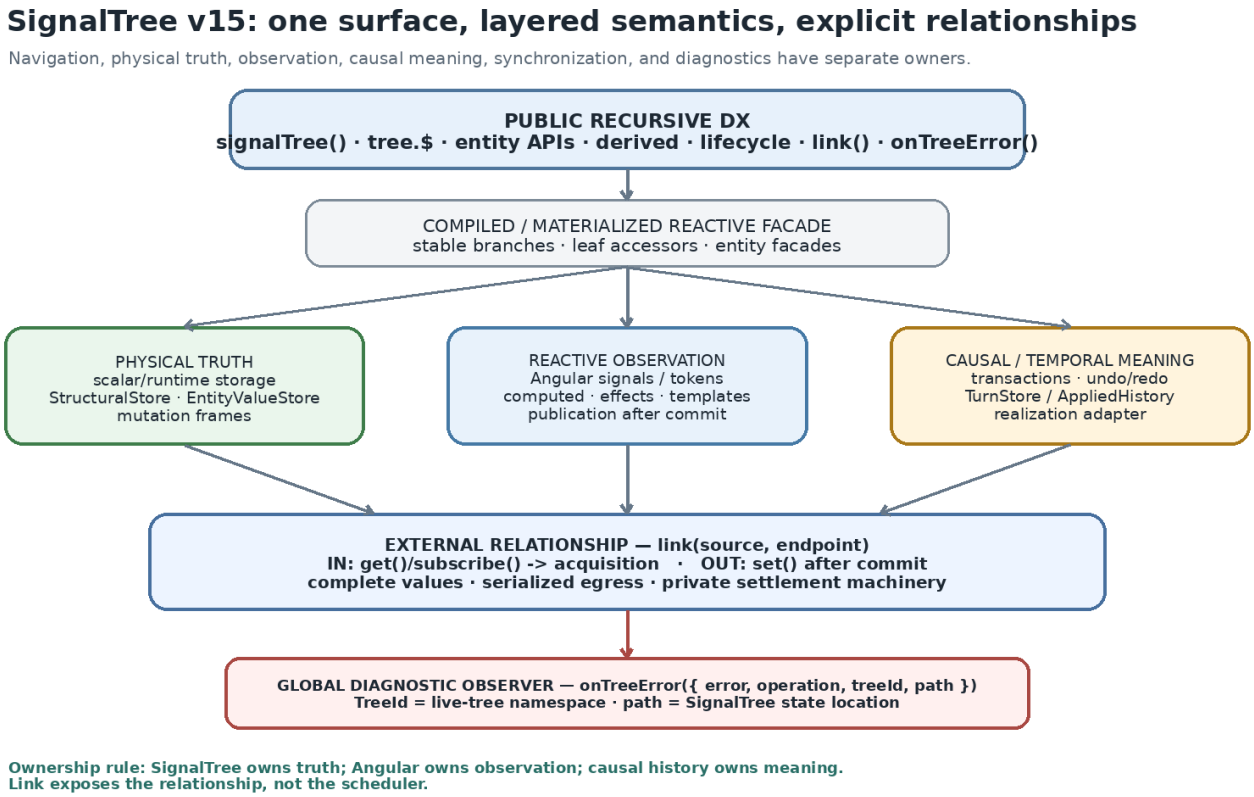
a benchmark result is not evidence until the harness itself survives falsification.

v15 also makes a deliberate public API break: chained enhancer composition through .with() is removed. Enhancers are declared up front in signalTree(state, { enhancers: [...] }), enabling whole-configuration validation, dependency ordering, truthful capability planning, one materialization boundary, and static restoration authority before runtime begins.

This paper explains what v15 is, how it works, why each major decision was made, what alternatives were rejected, what the current measurements actually support, and what remains open before release.

Executive summary

SignalTree v15 can be summarized as a transition from a reactive object-building library with accumulated historical machinery into a planned state engine with explicit semantic boundaries.



What it shows: the public recursive API sits above separate layers for physical truth, Angular observation, and causal/temporal meaning, with link() as an explicit external relationship and onTreeError as a tree-attributed diagnostic channel. **Comparison:** v15 does not ask Angular signals to simultaneously be storage, history, lifecycle identity, synchronization protocol, and diagnostics. **Takeaway:** the rewrite is primarily a separation-of-responsibilities architecture, not one exotic data structure.

At the public surface, SignalTree remains recognizably SignalTree: structured state is navigated through the recursive tree.$ facade, leaves are reactive and writable, entities use entityMap, enhancers add capabilities, derived state remains supported, and lifecycle methods remain part of the tree contract. Underneath, however, v15 has been systematically reworked around one constructor, one build plan, one physical representation, one capability dependency graph, and one materialization boundary.

A useful way to read the rest of the paper is as a before/after comparison. v15 does not replace the public mental model with a lower-level kernel API; it moves complexity behind a more explicit construction and lifecycle contract.

Concern	Earlier shape / failure mode	v15 shape	Engineering consequence
Enhancer composition	Chained .with() could finalize physical state before all enhancers were known	All enhancers declared in the constructor configuration	Planning can see the whole configuration before materialization
Capability plan	Legacy all-on plan made bare trees look causally capable	Resolved TreeBuildPlan from requested capabilities and enhancer dependencies	Optional machinery can be omitted truthfully
Entity bulk load	Eager generalized position/node machinery inflated setAll	Authoritative stores first; activation/observation machinery is lazy where semantics allow	Bulk population no longer pays for every possible future observer

Ordered collection projection

Permanent second ordered representation

Derive whole projections from authoritative storage when realized

More work on uncached all(), materially less permanent memory

Retired entity lifetime

Central records accumulated after retirement

Zero-owner subjects reclaim backing and are fully forgotten

No-history churn becomes asymptotically flat

Reactive identity optimization

Weak interning looked attractive from ordinary tests

Strong continuity retained where required; GC gate guards it

Correctness wins over attractive but unsafe memory savings

History / rollback

Physical restoration and causal meaning were easy to conflate

Authorship, realization, publication, and persistence are separate concepts

Rollback/undo can restore truth without pretending to author a new user mutation

Tree lifetime

Abandonment was implicitly treated like collection garbage collection

destroy() is an explicit tree ownership boundary

Tests, SSR, routes, and temporary stores need lifecycle teardown

External synchronization

Ad hoc external effects risked conflating commit timing, hydration, echo suppression, and failure

Public link(source, endpoint) relationship with explicit retrieve(), strong settled(), and dispose()

SignalTree exposes the relationship, not its private settlement machinery

Link value semantics

Patch/merge/comparator modes were plausible but unearned

Complete NaturalValue snapshots with the existing structural deepEqual rule

Collections cross the boundary as complete Row[]; internal granularity remains independent

Global error observation

Internal reporter events lacked tree attribution and carried dead taxonomy

Public onTreeError with { error, operation, treeId, path? }

Same-shaped trees are distinguishable without exposing reporter internals

Benchmarking

Single-run or single-GC observations could look architectural

Interleaved arms, quiescence, A/A controls, gate self-tests, quarantine

The harness itself must survive falsification before its numbers become claims

The most important closed decisions are:

Area	v15 decision	Why
Construction	Declarative enhancer configuration; .with() deleted	The complete enhancer set must be known before materialization for the build plan to be truthful.
Capability planning	Resolve dependencies once and materialize only required machinery	Physical support should correspond to actual configured capabilities rather than a legacy all-on plan.
Entity projection	Delete permanent MaterializedEntityProjection	It made uncached all() faster, but cost about 126 B/entity everywhere and did not help downstream fan-out.
Activation metadata	Realize subject activation tokens on demand; delete unearned subject-position transport	Removed a major setAll regression and large existence/transient overhead without losing semantics.
Entity reactive identity	Keep durable subject-scoped signal interning for live/restorable subjects	Weak interning passed ordinary tests but failed forced-GC correctness.
Subject identity	Key reuse creates a new subject lifetime	Old handles must never follow a fresh occupant of the same key.
Zero-owner retirement		

Reclaim value/signal immediately and forget the whole subject lifetime

A permanent tombstone ledger was falsified; stale-handle safety survives without it.

Transactions

Preserve both subject ID and scoped leaf address during rollback

Entity-field rollback had silently replaced whole rows with scalar field values.

Tree lifecycle

destroy() is a real ownership boundary

Abandoned write-active trees remain retained; destroyed trees do not accumulate.

External relationship

Ship one public link() API; keep settlement/notifier machinery private

Strong settlement, explicit retrieval, serialized egress, echo suppression, and disposal are proven without adding mode/status/retry APIs.

Link boundary

Exchange complete values; use one structural equality rule; no comparator/patch protocol

Fresh-but-equal values suppress echo; entity collections synchronize as complete Row[] snapshots.

Error observation

Publish onTreeError, TreeErrorEvent, and TreeId only after attribution repair

TreeId distinguishes live tree namespaces; path consistently names the SignalTree state location; internal report machinery stays private.

Evidence consolidation

Archive experiment harnesses once production conformance subsumes them

Comparison modes that could not detect a catastrophic production break were removed; the winning invariants remain in production-facing tests.

Metadata layout

Keep property metadata for now; sidecar deferred

Measured live-node delta is only about 6-7 B/entity, too small to justify conflating it with larger work.

Owned history reclamation

Open Step 8

timeTravel retains retired subjects and is not visible to the existing TurnStore-based eligibility assessor.

The result is not "SignalTree is always O(1)" and not "SignalTree keeps stable signal objects forever." The defensible statement is narrower and stronger:

SignalTree v15 is a fine-grained reactive state engine for structured TypeScript state. Entity and other subject-backed state can carry non-reused lifetime identity, so subject-scoped reactive references remain associated with the lifetime from which they were acquired rather than silently following a later occupant of the same address. Point mutations are localized to affected state and reactive dependents rather than inherently scaling with unrelated collection members. When a retired subject has no legal restoration owner, its central backing and lifetime bookkeeping can be reclaimed completely.

A separate tree-level contract also matters:

A bounded-lifetime SignalTree must be destroyed. Abandoning a write-active tree is not equivalent to destroy().

---

1. Scope, evidence, and claim discipline

1.1 What this paper covers

This whitepaper covers the v15 work as an engineering program rather than only the final API. It includes:

the architecture north-star that guided the rewrite;

public type and API characterization;

the physical state and entity representation work;

workload evidence and missing-capability audits;

the projection fork and its removal;

activation and subject-position cleanup;

reactive identity and forced-GC durability;

enhancer/capability planning and the declarative-construction migration;

subject retirement, zero-owner reclamation, and lifetime forgetting;

the transaction rollback defect uncovered during that work;

causal/history ownership boundaries and the still-open owned-history case;

the store-level destroy() lifecycle discriminator;

the current performance and memory baseline;

competitor benchmarks and their limits;

package/type/release-gate hardening;

rejected alternatives and corrected conclusions;

the remaining path to RC.

It does not present unfinished proposals as shipped behavior. Where an earlier architectural prototype was used to set direction but not necessarily implemented literally, it is labeled **NORTH-STAR**. Where a result is empirically established on the current v15 line, it is labeled **ESTABLISHED**. Where work is incomplete, it is labeled **OPEN** or **QUARANTINED**.

1.2 Evidence hierarchy

The v15 program intentionally ranked evidence by how directly it represents the product:

real application behavior and runtime traces;

published empirical studies and production engineering evidence;

observed developer workarounds and missing-capability signals;

source-level architectural audits;

competitor implementations;

microbenchmarks.

Microbenchmarks remain useful, but they come last because the workload model determines which measurements matter. A benchmark can tell us the cost of an operation; it cannot tell us how important that operation is in applications without an external workload model.

1.3 Observed, strategic, and inferred claims

The project repeatedly corrected claims that had mixed categories. For example, 28 production byId() call sites are observed. "Most application mutations are point mutations" is not observed merely because updateOne exists or

because it appears prominently in an API. Collection sizes of 10k-100k are a product envelope assumption, not evidence that typical production stores have those sizes.

The whitepaper therefore distinguishes:

Observed: measured directly in code, a production app, or a controlled falsifier.

Strategic assumption: deliberately chosen workload or product envelope.

Inference: architectural explanation consistent with measurements but not itself directly measured.

Open: not yet established.

1.4 The economic model changed during the work

An early framing tried to combine construction, operation cost, and memory into one "lifetime cost." That was corrected. Milliseconds do not repay megabytes unless an explicit utility function says they do.

The v15 economic model is a vector:

```
ARCHITECTURE PROFILE = {
  construction latency,
  steady-state CPU,
  operation latency,
  retained memory,
  peak/transient memory,
  developer coordination cost
}
```

Product decisions then apply budgets and priorities to that vector. This correction was load-bearing in the materialized-projection decision: a store performing many all() recomputations does not mathematically "recover" 12 MB of additional retained memory. It may justify the expenditure, but that is a product trade, not a numeric break-even.

1.5 Harnesses are evidence only after their own controls pass

The v15 process treated the benchmark harness itself as a system under test. This proved necessary multiple times:

a grep pipeline stripped file paths before filtering tests, inflating production call counts;

a workload harness rounded low event counts to zero, silently changing the intended workload;

sequential benchmark arms against stale builds produced a false performance regression;

a later run produced a false improvement in the other direction;

a one-gc() memory script under-measured retained heap and led to a false lifecycle explanation;

a forced-GC correctness bug passed the entire ordinary test suite;

a slope-gate self-test initially registered as blind because mutating only one of two conditions did not disable the verdict;

one current 100-field benchmark cell remains quarantined because the harness consumes far more memory than standalone reproductions and the cause is still unlocalized.

This is not incidental process detail. It is part of the v15 architecture story because several architecture decisions changed only after the measurement protocol became trustworthy.

---

2. Why v15 became an architectural release

2.1 The problem was not one slow function

The work that became v15 initially looked like a set of performance and correctness tasks. In practice the failures shared a deeper cause: too many distinct semantics were encoded through overlapping objects and lifecycle assumptions.

Examples included:

a generalized position-collection path materialized rich entity facades during setAll, turning an initialization operation into expensive per-member object work;

an entity projection duplicated ordered structural state without being used by production reads;

every public signalTree() used a legacy capability plan that said every optional capability existed;

chained .with() materialized the tree before the first enhancer was known;

reactive subject identity was intertwined with key identity strongly enough that weak retention looked safe until GC disproved it;

retirement state retained values for subjects that no configured feature could ever restore;

a transaction rollback classifier treated subjectId and a scoped leaf path as mutually exclusive even though entity-field writes require both.

The common solution was not "optimize harder." It was to make the semantic dimensions explicit and let each subsystem own only what it actually needs.

2.2 The architectural north-star

NORTH-STAR, not a claim that every low-level representation literally shipped. Early v15 kernel work established the direction:

SignalTree owns truth.
Angular owns observation.
Causal history owns meaning.

The model distinguished:

PositionId = semantic causal/topological identity
SlotIndex = physical storage address
SubjectId = structural/entity lifetime identity
key/path = logical address used by the public state namespace

The prototype direction explored compact state slots, versions, framework-neutral physical truth, Angular dependency tokens as an observation adapter, and atomic mutation frames. The key architectural point survived even where exact structures evolved: physical storage identity, semantic causal identity, structural subject identity, and public address must not be collapsed merely because they can coincide in one implementation.

2.3 Construction as compilation

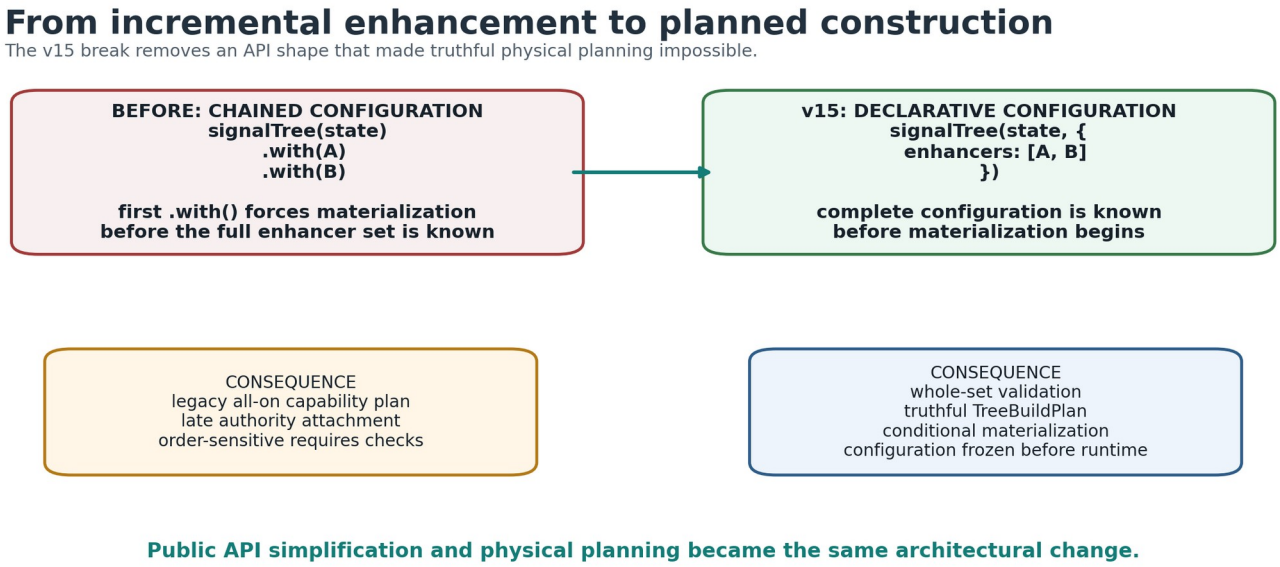
Another north-star principle also survived into the public v15 design: construction is compilation work that is allowed to be more expensive if it reduces steady-state work.

The desired lifecycle is:

configuration
  -> resolve requirements
  -> choose physical capabilities
  -> materialize stable access structures
  -> run cheap repeated operations

This is why the project evaluates construction separately from steady state instead of optimizing every line for startup microseconds.

2.4 One implementation, not legacy plus new



Public API simplification and physical planning became the same architectural change.

What it shows: chained enhancement and planned construction are not equivalent implementation details; they create different information boundaries. **Comparison:** the old path could materialize before seeing later enhancers, while v15 validates and plans from the entire enhancer set. **Takeaway:** deleting .with() made the public API break and the truthful-planning fix the same change.

A recurring release rule was that v15 should not ship as:

legacy implementation
+ correct new implementation
+ runtime exceptions and compatibility shims

The target became:

ONE constructor
ONE planning system
ONE physical representation
ONE capability dependency graph
ONE materialization boundary
ONE causal eligibility system

That principle ultimately drove the deletion of .with(), plannedSignalTree(), the legacy all-on build plan, and obsolete A/B tooling whose comparison no longer had two living arms.

2.5 Data-model survey: bounded fanout, not structural sharing by itself

Before the later entity and lifetime work, v15 ran an independent survey of persistent/immutable data structures to challenge the assumptions behind the rewrite. That work produced a result that shaped the rest of the program: **width is the expensive structural dimension; structural sharing is useful only when it bounds the amount of structure an operation must visit or copy.** [E26]

Representative measurements from that survey included:

reference-equality diffing on a plain 50k array was about 232 us, versus about 87 us for the existing deep comparison - 2.7x slower because both still scan 50,000 positions and the extra recursive/reference checks add overhead;

an equivalent trie-localized change could be identified in roughly 0.26 us because the structure bounds the search fanout;

an Immer update through a 15-level-deep object containing a 50k-wide array allocated about 392 KB, with approximately 99.6% of the bytes attributed to the wide array node and only 0.4% to the fifteen levels of depth;

immutable.List at 50k demonstrated the coupled benefit of bounded fanout in the research harness: roughly 97 ns update, 2.0 KB retained per version, constant-time root restore, and change localization that stayed nearly flat as N grew.

The conclusion was not "replace SignalTree with a persistent trie." Whole-state materialization/read ergonomics are in real tension with persistent-tree locality. Instead, the survey corrected the optimization target:

BAD TARGET
  structural sharing everywhere

BETTER TARGET
  do not force local work through wide nodes
  use bounded/local structures where the workload requires locality

This reinforced entityMap as the right abstraction for large mutable collections. It also killed a proposed shortcut: globally replacing deep equality with reference equality at leaf boundaries. Caller-provided arrays from HTTP/refetch paths do not preserve reference identity, so Object.is would turn value-equivalent refreshes into false changes. The remeasurement showed that even the best shallow array-leaf path remained dominated by wide-array copying; entity modeling, not equality tuning, was the real solution.

2.6 Slot/token feasibility: the implementation north-star was tested, not merely imagined

The early physical-kernel prototype separated committed SignalTree values from Angular observation tokens. It was not the final literal v15 representation, but it established that the ownership split was feasible.

The prototype demonstrated:

Angular computed/effect behavior over SignalTree-owned values;

unrelated-leaf invalidation isolation;

atomic pair visibility for multi-leaf commits;

stale-frame protection;

OnPush and zoneless template integration;

ordinary slot/token writes around 0.0005 ms in the small prototype;

an atomic two-leaf commit around 0.0013 ms. [E27]

Those numbers were feasibility signals, not release performance claims. Their architectural value was proving that Angular Signals did not have to be the storage owner in order for Angular reactivity to work. That result supported the durable v15 rule:

state truth SignalTree-owned
reactive observation Angular-adapted
causal meaning semantic layer above physical truth

Later v15 code uses concrete structures such as StructuralStore, EntityValueStore, mutation frames, and the realization adapter rather than requiring every field to live in the exact typed-array layout proposed by the prototype. The north-star survives as separation of responsibilities, not as a promise of a specific memory layout.

3. Public product model and TypeScript contract

3.1 The public model remains structured and recursive

The rewrite did not abandon SignalTree's core developer experience. A normal tree still represents structured TypeScript state and exposes recursive navigation:

```
const tree = signalTree({
  user: {
    profile: {
      name: 'Ada'
    }
  }
});

const name = tree.$.user.profile.name();
tree.$.user.profile.name.set('Grace');
```

The public hierarchy is tree-shaped even though the reactive dependency topology is a graph. Derived computations may depend on multiple branches; one leaf may have many consumers. "Tree" is therefore a product and state-namespace description, not a graph-theoretic claim about all runtime dependencies.

3.2 Branches and leaves are intentionally different

The v15 type-characterization matrix protects a subtle but important rule sometimes referred to in the engineering notes as **Rule 0d**:

a leaf is a callable writable reactive value;

a branch remains a structurally navigable accessor;

branches support whole-branch read/write/update call forms;

a branch does not become an Angular signal merely because its descendants are reactive leaves.

This is exactly the sort of type semantic that can disappear during API cleanup while every runtime test remains green. The type matrix therefore uses exact type equality and negative controls, not broad assignability alone.

3.3 The type matrix became a permanent semantic gate

Before deleting redundant public types, v15 first characterized what consumers can actually do. The matrix covers:

root object trees;

primitive tree type coherence;

nested access and Rule 0d;

marker-resolved surfaces;

entityMap typing;

enhancer-added methods;

lifecycle methods such as bind, destroy, and cleanup registration;

negative cross-state assignment controls;

the actual inferred return of signalTree(), not only a manually annotated SignalTree<T> value.

A key methodological choice was to assert consumer semantics, not freeze implementation decomposition. For example, tests deliberately did not require ISignalTree to remain a distinct public conceptual layer.

3.4 SignalTreeBase was deleted because it added no contract

SignalTreeBase<T> was character-identical to SignalTree<T>. Rather than deleting it because it "looked redundant," v15 built a falsifier:

Find any consumer or type contract where replacing SignalTreeBase<T> with SignalTree<T> changes expressible semantics, inference, assignability, or capability.

A universal generic identity check plus sampled rows found no distinction. Mutating the alias deliberately made the gate fail, proving the test was live. Only then was the alias removed. This sequence - characterize, falsify, delete - became the standard for public-surface cleanup.

3.5 Enhancer-added typing is now constructor-derived

Chained .with() previously accumulated types through intersection with each call's addition. v15 had to preserve that ergonomic guarantee when enhancers moved into configuration.

The new return type derives additions from the enhancer tuple:

```
const tree = signalTree(
  { count: 0 },
  {
    enhancers: [timeTravel(), batching()]
  }
);

tree.canUndo();
tree.batch(() => {});
```

A real inference trap surfaced during implementation: an enhancer with no additions could infer TAdded = unknown, and unknown absorbs a union. One no-op/probe enhancer could therefore erase every real addition beside it at the type level while runtime behavior remained correct. The fix maps the no-addition case to never, the union identity.

3.6 Cast removal exposed real product defects

The declarative migration removed hand-written casts in demo code because enhancer methods now arrive through the constructor's inferred return type. That exposed two DevTools inconsistencies:

exportDebugSession() existed at runtime and was asserted in tests but was absent from DevToolsMethods;

the enabled: false path did not implement it and threw when called.

Both were corrected, and public session types were exported so declaration-emitting consumers can name the returned structures. This is an example of a type-system cleanup uncovering runtime contract drift rather than merely changing syntax.

3.7 Public-surface and package discipline

The v15 work also treated exported names and packages as semantic liabilities that need evidence, not as harmless implementation leftovers. Public API inventory tools and external tarball consumers were used to distinguish:

symbols that are merely exported inside internal source modules;

symbols reachable from a package root/subpath;

symbols present in generated .d.ts output;

symbols actually usable from an external consumer project.

This distinction prevented internal testing hooks from being labeled semver-breaking simply because they used TypeScript export, and it supported deletion of redundant aliases such as SignalTreeBase and redundant composition helpers such as composeEnhancers.

A broader package audit also removed packages/surfaces that did not earn a release contract, while preserving the recursive core DX. The exact final publishable package inventory is treated as a release artifact and is still subject to the hardening gates; it is intentionally not used as a premise for the physical architecture.

The practical rule is:

Freeze what consumers can do, not how the implementation happens to be decomposed today.

That rule is why the type matrix focuses on call forms, marker resolution, entity access, enhancer additions, and lifecycle methods instead of pinning internal aliases or module boundaries.

3.8 External relationships are intentionally small

The final v15 external synchronization surface is deliberately narrow:

```
interface LinkEndpoint<T> {
  get?(): T | Promise<T>;
  set?(value: T): void | Promise<void>;
  subscribe?(next: (value: T) => void): () => void;
}

interface Link {
  retrieve(): Promise<void>;
  settled(): Promise<void>;
  dispose(): void;
}

link(source, endpoint): Link;
```

The handle has no public status, retry policy, error signal, mode switch, comparator, post-commit callback, or link identifier. Those additions were repeatedly considered and rejected because the behavior could be made correct through private settlement/notifier machinery and one generic error observer.

retrieve() is explicit rather than automatic hydration. settled() is whole-relationship idle: it waits for outbound work and in-flight retrievals, including work produced by acquisition, and disposal releases waiters while suppressing late acquisition.

3.9 The public error contract is attribution-first

The generic observer is:

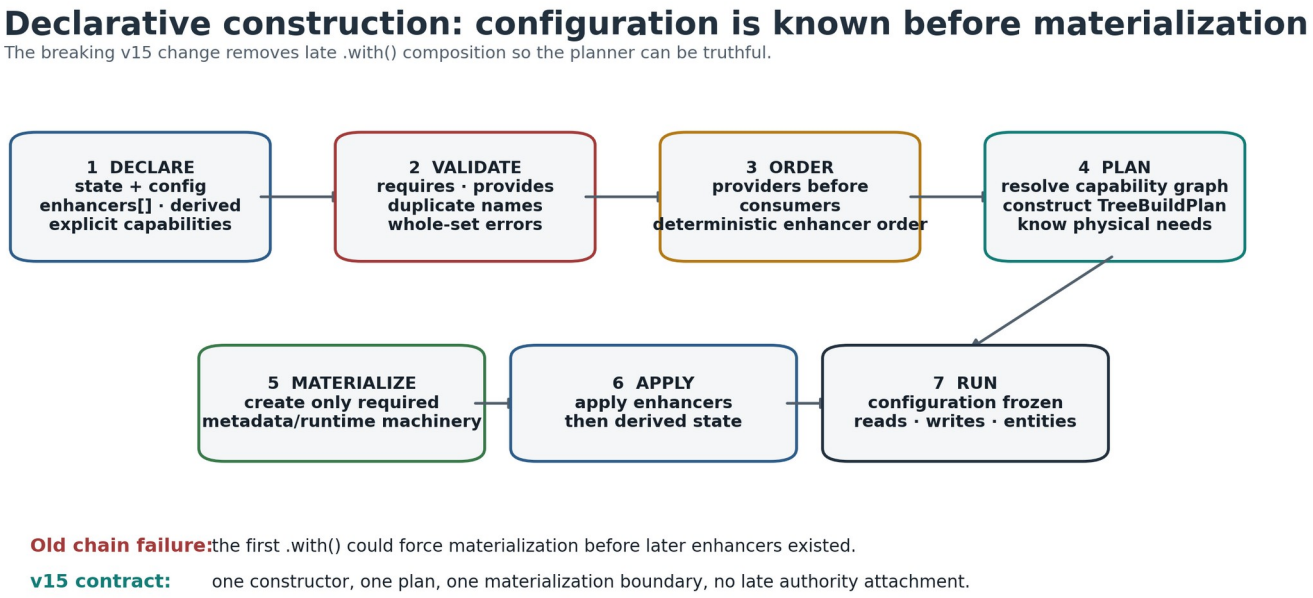
```
interface TreeErrorEvent {
  readonly error: unknown;
  readonly operation: string;
  readonly treeId: TreeId;
  readonly path?: string;
}

onTreeError(listener: (event: TreeErrorEvent) => void): () => void;
```

TreeId is an opaque, runtime-local correlation identifier for one live tree namespace. The runtime representation is currently numeric, but ordering, persistence, serialization identity, recreation, and cross-process stability are explicitly not part of the contract. The brand prevents arbitrary numbers from being accepted as TreeId; it does not turn the underlying number into a non-numeric runtime object.

Two dead fields were removed before publication. source was a mostly unproduced taxonomy that duplicated operation; detail had one producer and zero consumers. They were deleted from the delivered object rather than merely hidden from TypeScript because listeners receive the same runtime object the reporter is given.

path also received a final semantic repair before export: it now means the SignalTree state location for every producer. A stored() node reports its ownerPath, not its unrelated storage key. This lets (treeId, path) act as truthful runtime attribution without confusing location with identity.

4. Declarative construction and truthful capability planning



What it shows: the complete configure -> validate -> order -> plan -> materialize -> run pipeline. **How to read it:** each stage consumes facts established by the previous stage; runtime begins only after the configuration has been frozen. **Takeaway:** there is one construction path rather than a legacy path plus a planned path.

4.1 Why .with() had to go

This was not primarily a style preference.

The old public builder did something structurally fatal to truthful planning: .with() finalized/materialized tree markers **before** applying the enhancer. Therefore the physical build plan had already been chosen when the first enhancer became visible.

The result was a legacy public path with an effectively maximal plan. Bare trees carried machinery associated with causal or temporal features even when those features were not configured.

A second, unshipped plannedSignalTree() path already showed the right idea: collect enhancers, resolve capability requirements, build a plan, then materialize. But it was a staging implementation, absent from the public barrel and tree-shaken from the shipped build.

The architectural choice was therefore not to add a second public constructor. v15 absorbed the good planning machinery into signalTree() and deleted the duplicate paths.

4.2 Whole-configuration validation

The v15 constructor takes the complete enhancer set:

```
const tree = signalTree(state, {
  enhancers: [
    timeTravel(),
    transactions(),
    batching()
  ]
});
```

Because the complete set is known before construction, validation asks the correct question:

Is this configuration satisfiable?

rather than the old sequential question:

Has the provider already appeared in an earlier .with() call?

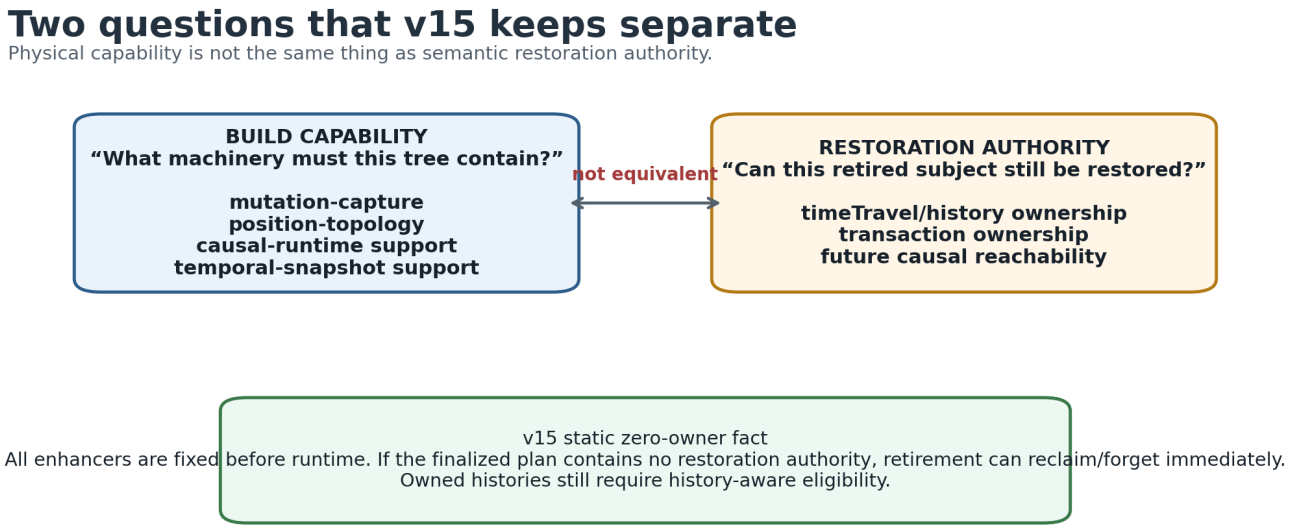
This allows a consumer to be declared before a provider in the array. Requirements are checked against the union of configured capabilities; dependency ordering runs providers first. Unsatisfied requirements remain hard errors and can be reported together. Duplicate named enhancers are also rejected under the chosen v15 configuration contract.

4.3 Capability graph

The existing capability dependency graph became authoritative:

```
mutation-capture      -> []
position-topology     -> []
causal-runtime        -> [mutation-capture, position-topology]
temporal-snapshots    -> []
```

TreeBuildPlan.has(capability) answers what physical machinery the tree needs. It does **not** answer whether a particular retired subject is still semantically restorable.



What it shows: physical support and semantic restoration rights answer different questions. **Comparison:** causal-runtime can mean machinery exists; it does not by itself prove a particular retired subject still has a legal restore path. **Takeaway:** zero-owner reclaim can use the static finalized configuration, while history-owned reclaim still needs causal eligibility.

4.4 Build capability and runtime authority are intentionally distinct

This distinction became one of the most important semantic corrections in v15.

A tree may physically contain causal runtime support because it was explicitly requested, even if no particular time-travel enhancer is attached. Conversely, reclaiming a retired subject requires knowing whether any semantic restoration path still owns it, not simply whether some broad physical capability exists.

The safe rules are:

TreeBuildPlan
  -> what machinery was materialized

RuntimeTreePlan / causal eligibility
  -> whether restoration authority exists

subject-level eligibility
  -> whether this specific retired lifetime is still reachable by restoration

Declarative construction makes the first authority fact static: enhancers cannot appear after runtime begins. This is what made zero-owner reclamation safely decidable at the retirement boundary.

4.5 The phase model

The v15 lifecycle is conceptually:

CONFIGURE
  state + enhancers + derived + explicit capabilities

FINALIZE / MATERIALIZE
  validate
  resolve ordering
  resolve capabilities
  create physical machinery
  apply enhancers
  install derived state

RUN
  reads, writes, entity operations, transactions, history

No public .build() is required. No live tree can later gain an enhancer through .with().

4.6 Blast-radius evidence

Before committing to the break, a reliable throwing probe measured what actually depended on .with() after materialization. The final blast radius was small: six spec files and eleven tests, concentrated in enhancer protocol semantics. Those tests were classified rather than blindly preserved. The meaningful guarantees - method forwarding, enhancer atomicity, dependency validation, restoration semantics - were rewritten around whole-configuration construction. The obsolete sequencing guarantees were intentionally removed.

The actual repository migration was larger mechanically because docs, examples, and ordinary enhancer use all had to move to enhancers: [...]. The semantic blast radius, however, was narrow.

4.7 Deliberate bundle cost

Truthful construction added code to every bare tree bundle because enhancer order resolution and configuration validation are on the mandatory constructor path. The measured cost was approximately:

+0.47 KB gzip production;

+0.49 KB development.

The resolver contributed about 851 B and validation about 817 B; deleting the old canonicalWith path recovered about 432 B. A runtime if (enhancers.length === 0) cannot restore tree-shaking because tree-shaking is static.

The project accepted this cost explicitly rather than reintroducing a second construction path to save hundreds of bytes.

---

5. Physical state architecture and the commit seam

5.1 What v15 means by "physical truth"

SignalTree's public state model is hierarchical, but v15 increasingly treats the runtime underneath as a state engine rather than as a recursively copied root object.

The concrete entity system uses authoritative structural/value stores and mutation frames. Earlier slot/token prototypes also established a broader north-star: stable public accessors should be able to resolve or cache physical locations so repeated reads and writes do not need to parse paths, traverse JSON, or reconstruct unrelated state.

The most durable rule is not a particular array layout. It is ownership:

Physical truth has one authoritative home, and multi-position mutation becomes coherent at an explicit frame/commit boundary.

5.2 Position, slot, subject, and address

The architecture keeps four identity categories conceptually separate:

Concept

Meaning

Reuse?

Typical owner

logical key/path

where state is addressed publicly

yes

public namespace

SubjectId

one entity/structural lifetime

no

entity structural semantics

PositionId

semantic causal/topological identity

stable semantic identity

causal/realization layer

SlotIndex

physical storage address

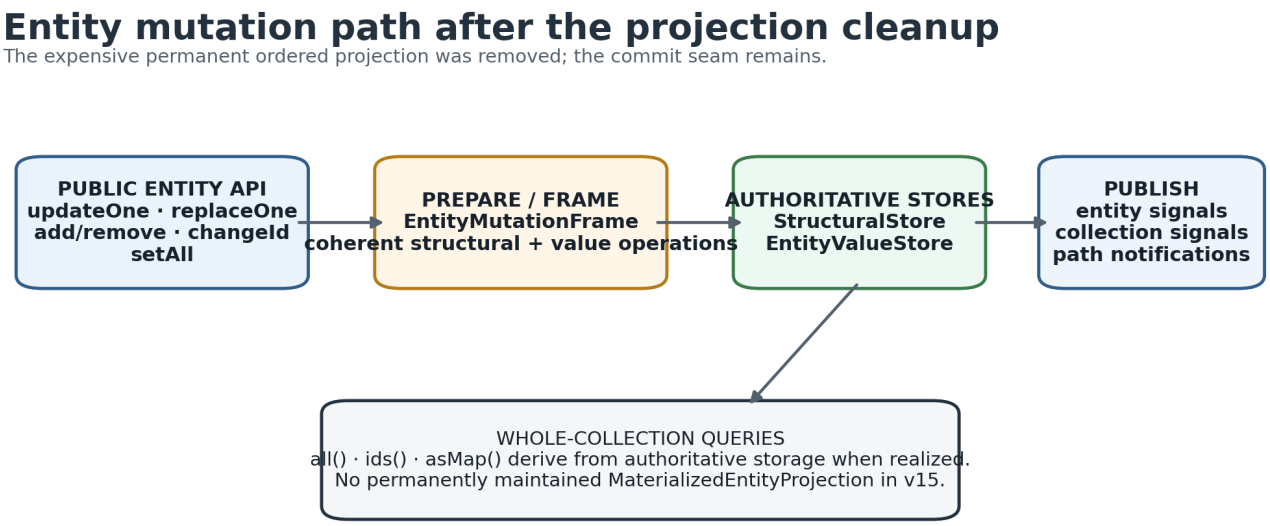
implementation-dependent

physical kernel

Even if a prototype maps a PositionId to the same integer as a slot, v15 does not make that equivalence a contract.

5.3 Atomic mutation frames

The physical design preserves a preparation/commit seam. For entities:



What it shows: entity operations become coherent through a mutation-frame boundary over authoritative structural/value stores. **Comparison:** v15 keeps the commit seam but removes the permanent duplicate ordered projection. **Takeaway:** removing a representation did not require collapsing the architecture that made later rollback/reclamation work possible.

A mutation may need to change more than one physical fact - values, active-key topology, subject state, rekey mappings, restoration placement, and reactive publication. An EntityMutationFrame stages coherent operations and commits them against authoritative stores.

The important retained abstraction is:

authoritative storage
    |
    v
mutation frame / commit boundary
    |
    v
query + reactive publication

This seam survived even when the MaterializedEntityProjection hanging off it was deleted. The architecture preserves the place where a change becomes coherent without preserving every old representation attached to that place.

5.4 Publication is downstream of committed truth

The broader v15 design differentiates:

LANDED WRITE physical truth changed
SEMANTIC MUTATION change participates in SignalTree's mutation semantics
CAUSAL AUTHORSHIP a turn/history authority authored it
PUBLICATION reactive observers are notified
CONSEQUENCE persistence/devtools/etc. observe committed outcome

These are dimensions, not synonyms. Rollback and restoration can change physical truth without creating a new user-authored causal turn. Persistence can observe a committed result without owning mutation authority.

This decomposition avoids the historical tendency to treat every write as one universal event object with every possible semantic role attached.

---

6. Workload evidence before representation decisions

6.1 TruckTrax production audit

A production application audit was used to constrain the entity workload model. After correcting a grep bug that had included tests, the authoritative production-only static counts on the v14.0.0 upgrade branch were:

API

production call sites

entityMap(...) declarations

7

.all()

44

.byId(

28

.ids()

0

.asMap()

0

.where(

0

The same profile appeared on the application's main branch, making it at least stable across that upgrade interval. These counts are **API-shape evidence**, not dynamic runtime frequency. [E01]

Nineteen of the forty-four all() reads occurred inside computed() in relevant derived tiers, and several collections had multiple reactive whole-collection consumers. That established that whole projections are not merely cold/debug paths.

6.2 Raw .all() counts overstated whole-collection intent

Ten of the forty-four all() call sites were all().find(...) used to find one entity by non-primary business keys such as projectExternalId, customerExternalId, externalId, or normalized name. SignalTree had no secondary-index/alternate-key lookup capability for those cases.

Therefore at least 23% of the apparent whole-collection reads were actually point-intent operations forced through a scan by a missing capability. The correct conclusion was not "whole reads beat point reads 44:28." It was:

point access is first-class;

whole projections are also first-class;

API shape can disguise intent when a primitive is missing.

6.3 Exact replacement exposed a second semantic gap

TruckTrax also contained a pattern like:

```
setAll(all().map(e => e.id === incoming.id ? incoming : e));
```

At first glance this looked like a point update implemented as whole-collection read/modify/write. Source inspection showed the developer was deliberately avoiding updateOne/upsertOne because those merge. The application needed exact replacement so properties absent from a server/rollback snapshot would actually disappear.

At the pinned SignalTree version, replaceOne did not yet exist. It appeared in 14.1.1. The workaround was therefore correct at the time and could not be counted as evidence that applications prefer whole reconstruction for single-row updates.

This produced a durable API rule:

Patch/merge and exact replacement are different semantics and must remain different operations.

6.4 Assumption ledger

The corrected workload assumptions used by later v15 experiments were:

Point reads are first-class.

Whole-collection projections are first-class.

Reactive projection fan-out occurs in real applications.

Mutation APIs must distinguish merge/patch from exact replacement.

Small mutations should not require collection-wide reconstruction.

Permanent collection-wide optimization cost must be justified by repeated workload or broader utility.

Transactions/rollback/reconciliation require exact restoration semantics.

Application workarounds must be audited for missing primitives before being treated as workload evidence.

The project deliberately did **not** claim a global distribution of collection sizes or mutation ratios without runtime telemetry.

7. Removing the major setAll regression

7.1 Symptom

A v15 development state had made setAll dramatically slower than the v14 baseline. Representative early measurements showed the same linear shape but much worse per-member constant cost:

N

v14-ish path

regressed v15 path

100

~0.147 ms

~1.100 ms

1,000

~0.233 ms

~9.964 ms

10,000

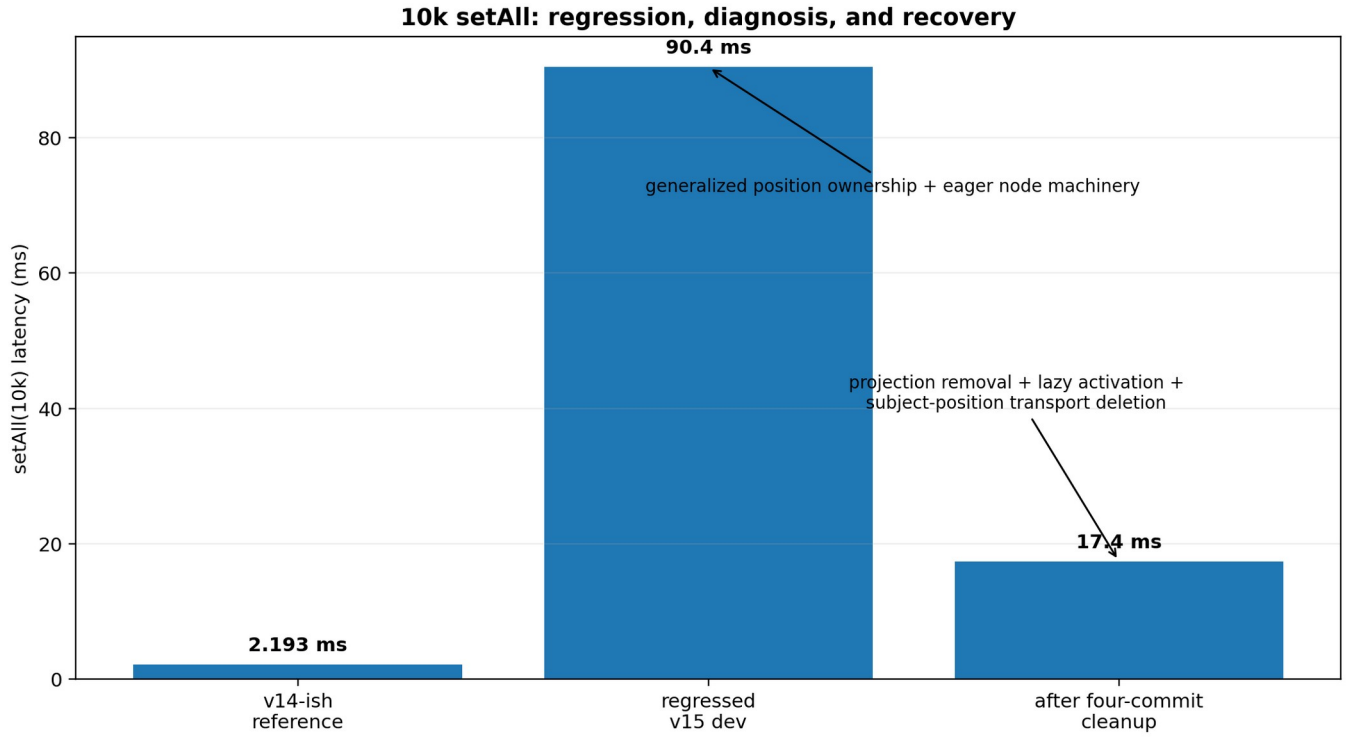
~2.193 ms

~90.436 ms

50,000

~12.699 ms

~528.467 ms



What it shows: the 10k bulk-load regression was real and very large; the subsequent cleanup recovered most of it. **Comparison:** ~90.4 ms in the regressed development state fell to ~17.4 ms after deleting unearned/eager machinery; the v14-ish reference was ~2.19 ms. **Takeaway:** v15 preferred a correct new architecture with a known remaining gap over restoring the old implementation just to recover one number.

The graph deliberately separates the old reference, the regressed v15 development state, and the post-cleanup v15 architecture. The cleanup did not simply restore the old implementation; it recovered most of the lost performance while preserving the new representation and lifecycle boundaries.

Profiling implicated generalized position ownership discovery and entity-node creation during bulk load. The architecture was effectively constructing rich facade/position machinery for every member during an operation that should mostly populate authoritative stores.

7.2 Subject-position transport did not earn its cost

The investigation asked what semantic function the eager subject-position transport actually served. The conclusion was that the general transport was not required on the hot path. It was deleted rather than merely optimized.

7.3 Activation tokens became lazy

Subject activation tokens also moved from eager per-subject construction to on-demand realization. This preserved functionality for consumers that actually need activation information while removing permanent/eager cost for untouched subjects.

7.4 Four-commit outcome

Across the projection removal, activation-token change, subject-position deletion, and benchmark-method repair, the 10k picture moved roughly from:

	setAll(10k)	base B/entity	same-turn heap
before	90.4 ms	1,173 B	59.9 MB
after	17.4 ms	486 B	~15 MB

Existence overhead fell approximately 718 -> 158 -> 31 B/entity through the sequence. [E02]

This was an important example of v15's design standard: do not optimize an expensive mechanism until its function has first earned the right to exist.

---

8. The materialized entity projection fork

8.1 What existed

entityMap maintained a MaterializedEntityProjection: a Map plus its own doubly linked list, parallel to ordering information already maintained by StructuralStore.

Production query paths did not use it; they derived results from authoritative storage. That made the structure suspicious, but "unused" does not mean "useless." The project first wired it into reads and measured the best case before considering deletion.

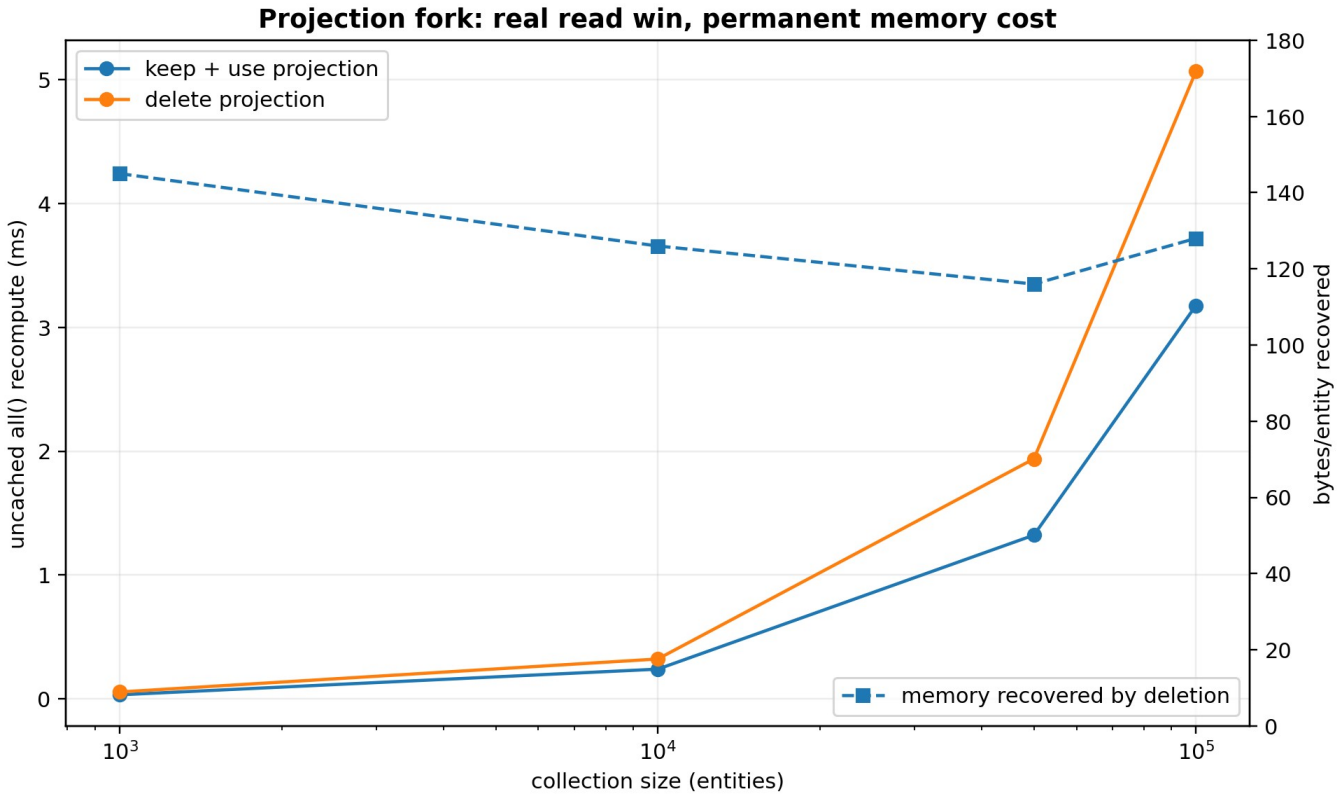
8.2 The three-way fork

The real options were:

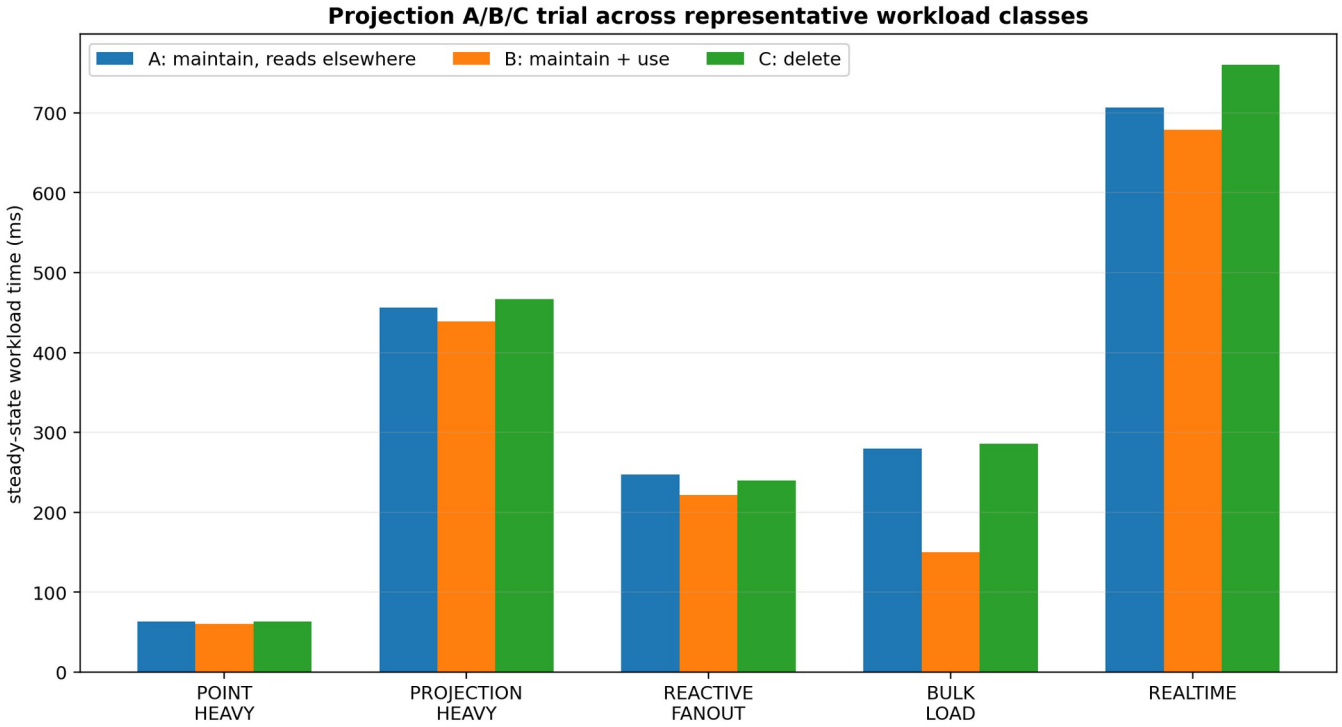
A. maintain projection; reads derive elsewhere
B. maintain projection; reads use projection
C. delete projection entirely

A was dominated by B: if the permanent structure stays, using it for reads captures a real benefit at almost no additional cost. The architectural question was therefore B versus C.

8.3 Measured trade



What it shows: keeping the projection makes uncached all() faster, and deleting it recovers roughly 116-145 B/entity. **Comparison:** the speed advantage grows with collection size, but so does the absolute permanent memory bill. **Takeaway:** v15 accepts the slower all() because the no-projection path remains inside the 100k performance guard and avoids charging every entity for one projection optimization.



What it shows: candidate B wins clearly on some workload fixtures but not universally. **Comparison:** the projection-heavy and bulk-load fixtures favor keep+use; point-heavy, fan-out, and realtime differences overlap run spread more heavily. **Takeaway:** the final decision used both workload behavior and permanent memory, not the best-looking microbenchmark.

The B/C comparison measured:

N

keep+use all()

delete all()

delete memory recovery

1,000

0.0319 ms

0.0546 ms

145 B/entity

10,000

0.2391 ms

0.3211 ms

126 B/entity

50,000

1.3218 ms

1.9379 ms

116 B/entity

100,000

3.1761 ms

5.0675 ms

128 B/entity

The read win was real and outside ordinary benchmark spread. Mutation deltas were sub-microsecond/noise at the per-operation harness scale. [E03]

8.4 Why deletion still won

v15 deleted the projection for five reasons:

Its benefit was concentrated in one operation: uncached whole-collection reconstruction.

The absolute no-projection cost remained inside the chosen performance envelope.

The memory cost was permanent for every entity, whether the collection was projection-heavy or not.

Angular computed is lazy and shared. With five derived consumers, the cost shape is closer to one shared all() reconstruction plus five consumer-specific traversals, not five independent all() rebuilds.

Future incremental derivation would likely need mutation deltas/change frames, not a permanent snapshot of current ordered entities.

A benchmark-machine release guard was recorded: 100k uncached all() should remain under 10 ms. The no-projection candidate measured about 4.87-5.07 ms in the relevant harness, so it passed with margin. This guard is not a universal consumer-hardware SLA.

8.5 What was removed

The concept was removed, not left as dead plumbing. The cleanup deleted the projection module, EntityMutationFrame.project(), projection-specific instruction arrays/types, rebuild flags, test hooks, shapeless generics that existed only for projection, and dead counters.

The architectural seam remained: authoritative stores -> mutation frame -> query/publication.

---

9. Entity reactive identity and the forced-GC falsifier

9.1 The function under test

Observed entity rows expose subject-scoped reactive access. Internally, v15 used a strong map from subject ID to an entity signal, created lazily when a row is observed.

That looked expensive after a full read. The question was whether the strong retention represented a correctness requirement or merely an implementation convenience.

9.2 Null 1: do not intern

Returning a fresh signal for every read broke established live-observation semantics and caused dozens of ordinary test failures. That established that some form of subject-stable observation identity was required while the subject is live/observable.

9.3 Null 2: weak interning

A much more attractive candidate replaced strong entries with WeakRef plus FinalizationRegistry. It produced major memory improvements and, critically, **all 1,791 ordinary core tests passed**.

Representative gains included:

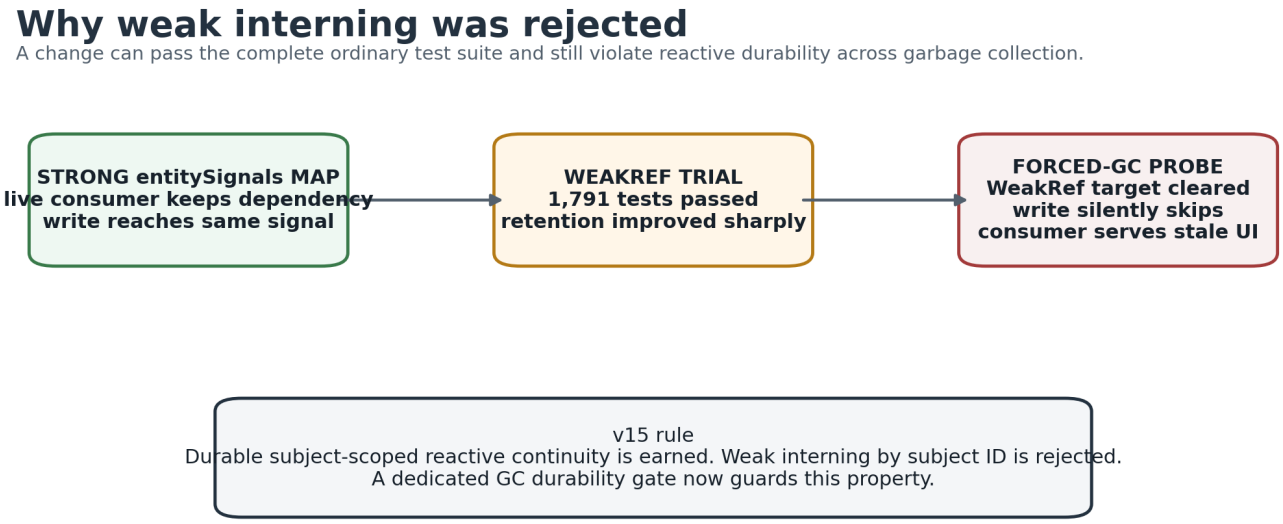
post-read transient residue: about 1,054 -> 498 B/entity;

no-history observed churn: about 798 -> 249 B/retired;

time-travel observed churn: about 1,859 -> 1,311 B/retired.

If the project had relied on the ordinary suite alone, weak interning would have looked like a clear win.

9.4 Forced GC disproved it



What it shows: an optimization can pass 1,791 ordinary tests yet still be wrong after GC. **Comparison:** weak interning reduced retention, but a live computed could later serve stale data because the signal target had been collected. **Takeaway:** subject-scoped reactive durability is an earned semantic requirement; the weak representation was rejected.

The decisive probe created a live computed consumer, stopped reading it temporarily, forced repeated GC and allocation pressure, mutated the underlying entity, then read the consumer again.

With weak interning, the underlying entity signal could be collected. Angular's dependency relationship did not guarantee that the weak target itself remained strongly reachable. The subsequent write found the cleared weak entry and silently skipped publication. The consumer served stale data with no error.

The committed strong-map implementation passed the same probe.

This produced two separate conclusions:

DURABLE SUBJECT-SCOPED REACTIVE CONTINUITY EARNED
WEAKREF INTERNING BY SUBJECT ID REJECTED

9.5 The GC gate became part of the release system

tools/check-signal-identity-durability.mjs was promoted to a gate because ordinary tests structurally cannot validate GC reachability. It checks:

a live consumer still invalidates after forced GC and pressure;

a held reference survives remove -> undo of the same subject;

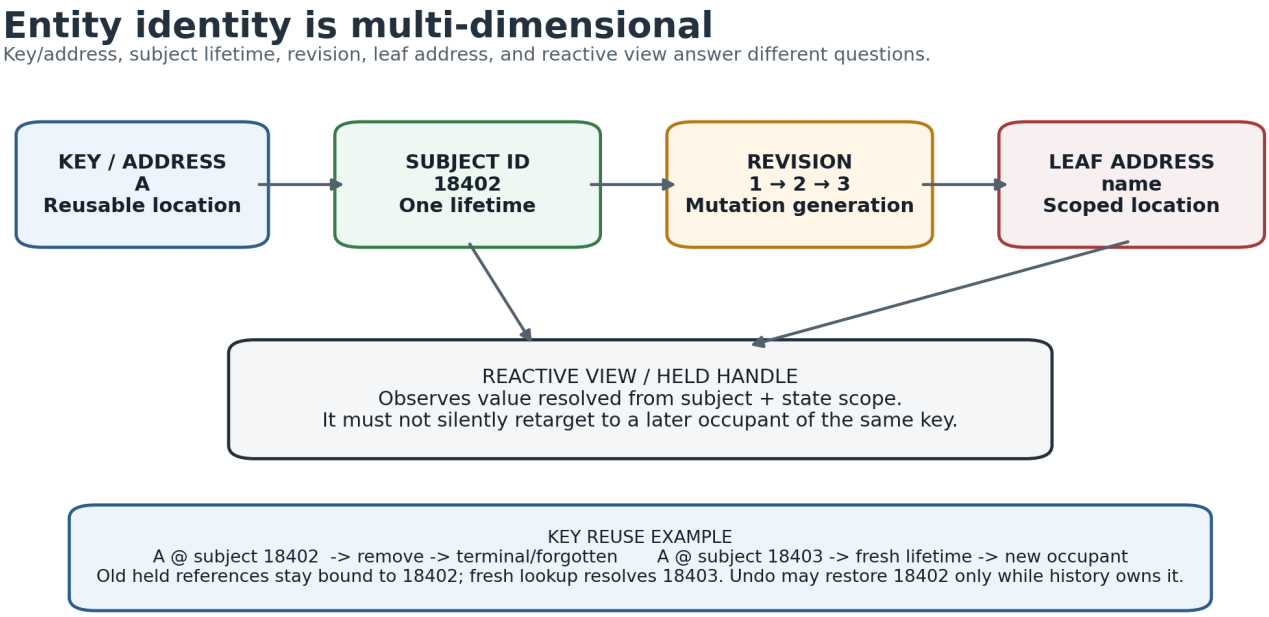
a held reference does **not** follow a fresh subject at a reused key;

independent consumers all invalidate.

A self-test proves the gate can fail. The project records honestly that a simple "never intern" mutation is also caught by the ordinary suite; the unique value of the forced-GC gate is the weak-interning failure mode.

---

10. Subject lifetime identity



What it shows: reusable key, non-reused subject lifetime, revision, scoped leaf address, and reactive view are distinct identities. **Comparison:** recreating key A does not recreate the prior subject; rollback of A.name needs both subject and leaf scope. **Takeaway:** many v15 bugs disappeared once identity questions stopped being represented as one path/string/object.

10.1 Key identity is not lifetime identity

For an entity collection, a business key identifies the current occupant of an address. It does not identify one continuous lifetime.

key A -> subject 18402 -> remove
key A -> subject 18403 -> later fresh add

Those are different subjects even if every field value is identical. Subject IDs are not JavaScript object identities; they are logical lifetime identities.

10.2 Revisions are not subjects

A revision answers which mutation generation of a subject is current. It does not create a new subject. This distinction allows stale-frame protection and mutation ordering without treating every update as a new entity lifetime.

10.3 Held handles are subject-scoped

A handle acquired while A refers to subject 18402 means "this subject," not "whatever entity eventually occupies key A." Therefore:

held old A -> remove -> undefined
fresh add A/18403 -> fresh lookup sees new row
held old A -> remains undefined

By contrast, undo may restore 18402 while history still owns 18402. That is restoration of the same lifetime, not key-based resurrection.

10.4 Subject identity is a semantic rule, not a demand for permanent tombstones

One of v15's most important later corrections was discovering that stale-handle safety does **not** require central metadata for every retired subject forever. A held terminal reference can remain semantically isolated even after the central subject record is completely forgotten, provided no legal SignalTree operation can target or restore that retired lifetime.

This distinction enabled the final zero-owner reclamation result.

---

11. Retired-subject churn and zero-owner reclamation

11.1 The original churn problem

With 1,000 live rows held constant and a completely new key generation inserted each round, a no-history tree showed linear retained growth:

rounds

retired subjects

growth

B/retired

10

10,000

2.75 MB

288

50

50,000

11.88 MB

249

100

100,000

22.47 MB

236

250

250,000

52.45 MB

220

Nothing referenced those retired subjects and no history owner existed, yet the store lifetime accumulated their value/lifetime records.

11.2 Decomposition showed half was immediately reclaimable

Using SignalTree's own internal reclamation path, not a bypass patch, the same 50k retired-subject fixture fell from about 249 B/retired to 119 B/retired. Roughly 130 B/retired - about 52% - was the retained entity value itself. Every candidate was reclaimable and the live collection remained correct.

This proved that the reclamation machinery already existed and that the missing piece was production eligibility/wiring.

11.3 Why declarative construction unlocked automatic zero-owner reclaim

Under chained .with(), "does this tree have a restoration owner?" could change later. A no-history tree could theoretically gain timeTravel after retirement. That made construction-time authority insufficient.

After .with() was deleted, the enhancer set became immutable before the first runtime write. The absence of restoration authority became a static fact.

The resulting rule is deliberately narrow:

```
NO restoration authority
  -> no transaction/history path can restore a retired subject
  -> release retained value backing and subject signal at retirement

restoration authority exists
  -> do not use zero-owner fast path
  -> owned/history eligibility is a separate problem
```

The new path was wired into removeOne, removeMany, setAll, and clear.

11.4 First-stage automatic reclamation result

The matched churn arms changed as follows:

arm

before

after zero-owner value reclamation

no history

249 B/retired

117 B/retired

no history + reads

797 B/retired

117 B/retired

time travel

1,310 B/retired

1,310 B/retired

time travel + reads

1,859 B/retired

1,859 B/retired

The time-travel controls remaining byte-identical is what makes the no-history result trustworthy; unconditional reclamation would have produced the same no-history win while corrupting undo.

The surprising result was the observed-row arm collapsing all the way onto the unobserved arm. Deleting the retired subject's entitySignals entry removed the entire observation-specific residue. Once a zero-owner row retires, previous observation no longer adds central retention.

11.5 The first fix was intentionally recorded as incomplete

The pre-registered criterion was not "make bytes smaller." It was "no-history retirement stops scaling with the number of historical subjects." At 50 and 150 rounds the residual remained roughly 117-140 B/retired. The shape was still linear, so the document recorded **FAIL**, despite the large reduction.

The remaining cost was the subject lifetime record, revision entry, and Map overhead.

---

12. Falsifying the permanent lifetime ledger

12.1 The semantic question

The remaining 117 B/retired appeared to support stale-handle isolation. The key question became:

Does stale-handle isolation require permanent central tombstone/revision records, or only enough semantic state in any reference that remains externally reachable?

The first null was deliberately aggressive: for a zero-owner retirement, delete the entire central subject lifetime/revision record and prove the public semantics.

12.2 The null was not refuted

The lifetime-ledger falsifier showed that old handles remained isolated without a permanent central tombstone. The project therefore flipped the default: zero-owner retirement now forgets the entire subject lifetime after terminal retirement.

Commit sequence around the result included:

a dedicated falsifier commit establishing the null;

feat(core)!: forget the whole subject at a zero-owner retirement;

a separate transaction correctness fix uncovered by the control suite.

12.3 A subtle resurrection bug was caught before the flip

During the trial, one retirement sequence performed:

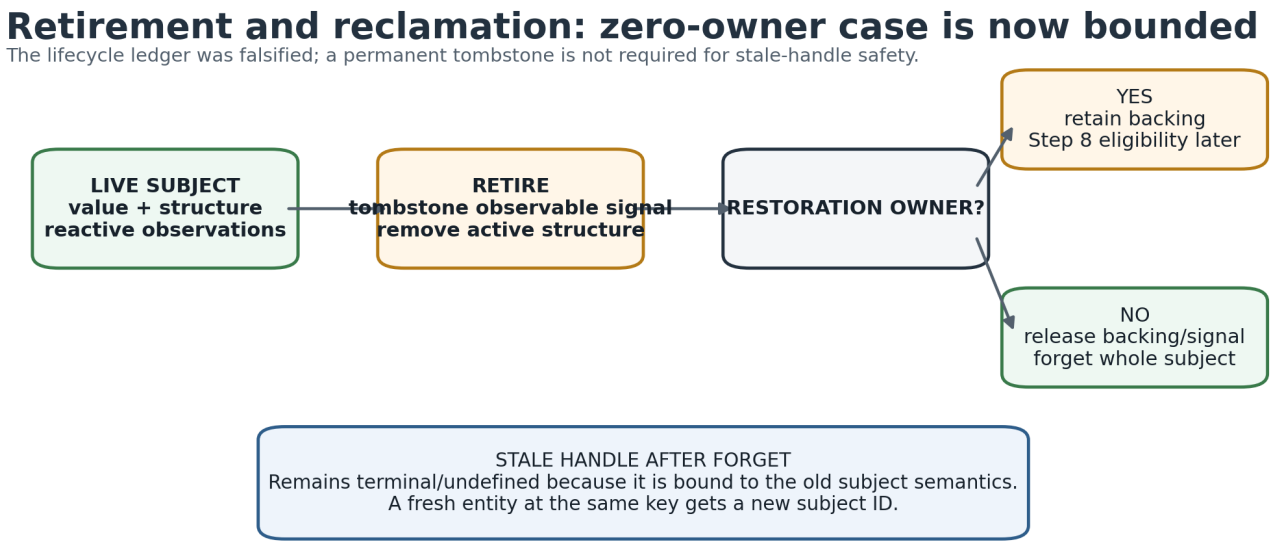
forget subject
  -> publishSubjectPhysicalChange
  -> bumpSubjectRevision
  -> recreate subjectRevisions entry

That would have undermined the whole asymptotic claim by re-interning metadata after the final boundary. Tests were added to assert the subject remains forgotten both at the end of retirement and after unrelated churn.

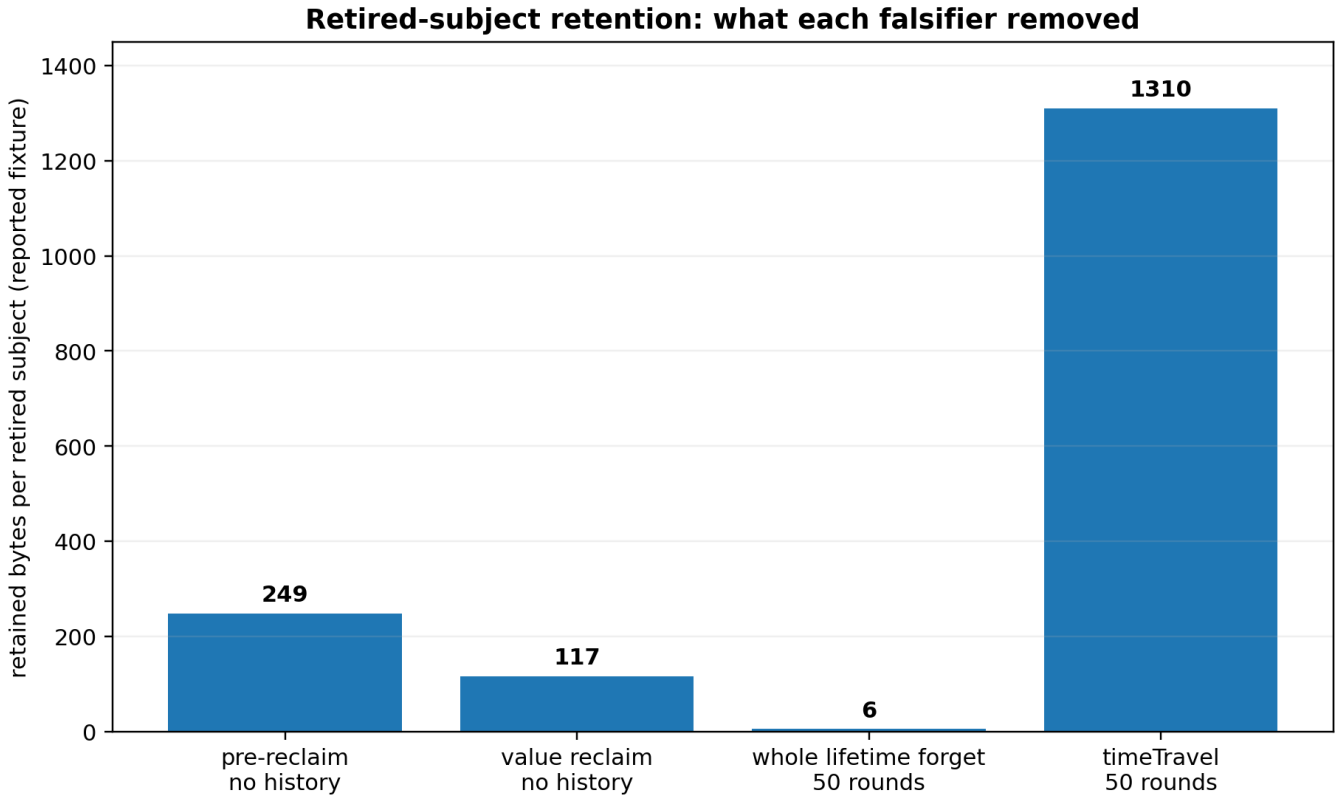
The resulting invariant is:

Once a subject crosses the final zero-owner reclamation boundary, no later step in that retirement path may recreate subject-scoped central metadata.

12.4 Final zero-owner asymptote



What it shows: retirement forks on restoration ownership. **Comparison:** a zero-owner subject can be fully forgotten immediately; a history-owned subject must retain backing until no restore path remains. **Takeaway:** Step 8 is not a redo of zero-owner reclamation - it solves the owned branch of the diagram.



What it shows: successive falsifiers removed observation residue, retained value, and finally the permanent zero-owner lifetime ledger. **Comparison:** no-history retirement moved from ~249 B/retired to ~117 B and then to a flat single-digit-noise asymptote; time-travel ownership remains intentionally expensive. **Takeaway:** the result is about slope/asymptotic growth, not celebrating a literal 6 B sample.

After whole-lifetime forgetting:

fixture

retained growth

reported B/retired

50 rounds, no history

~0.30 MB

~6 B

150 rounds, no history

~-0.83 MB

~-6 B

The literal small positive/negative values are measurement noise around a flat asymptote. The important result is that tripling retirements no longer scales total retained memory.

The dedicated slope gate compares 50 and 150 rounds and fails if the total begins scaling again. Its self-test rejects the old pre-fix table.

The final semantic disposition is:

STALE-HANDLE ISOLATION EARNED
PERMANENT TOMBSTONE LEDGER REJECTED
ZERO-OWNER RECLAMATION SHIPPED
ZERO-OWNER LIFETIME FORGET DEFAULT

12.5 What externally held stale references mean after forgetting

Central lifetime bookkeeping and external semantic reference durability are now deliberately different concerns.

A held old reference must remain terminal and must not follow a new occupant. But SignalTree does not need to retain a central tombstone merely because that reference object still exists. The reference can preserve its terminal semantics without keeping the retired lifetime targetable by the store.

This is why "external reachability" should not be interpreted as a requirement that central subject maps retain every externally held historical lifetime.

---

13. Owned history: the remaining reclamation problem

13.1 Zero-owner and owned retirement are different problems

The zero-owner case is statically decidable because declarative construction proves no restoration authority can ever appear.

The owned case cannot use that shortcut. If history or a transaction can restore a subject, deleting its backing is data loss.

After the zero-owner work, time-travel churn still grows substantially:

about 1,310 B/retired at 50 rounds;

about 1,407 B/retired at 150 rounds.

This is the next real lifecycle question, not a reason to weaken the zero-owner result.

13.2 Existing coordinator and the visibility gap

SignalTree already contains subject-reclamation-coordinator machinery and runPhysicalMaintenance. Its eligibility logic understands pending turns, confirmed turns, applied history, and redo state through TurnStore/AppliedHistory.

The critical mismatch is that timeTravel does not use TurnStore; it maintains its own history while depending primarily on the realization port. Therefore a TurnStore-only assessor can incorrectly report a time-travel-retired subject as unowned even when undo can still restore it.

This is why v15 does **not** simply run general maintenance at every retirement.

13.3 History participation scope narrowed before reclamation

A later history-scope audit separated restoration eligibility from whole-tree diagnostic history. The measured direction is operation/causal-turn scoped restoration eligibility with one restoration authority, rather than location-filtered partial reversal. Ordinary same-turn writes to multiple branches coalesce into one causal turn, so undoing only the "historical-looking" locations would partially reverse authored intent. Realizations themselves already acquire no restoration-history entry or claim ownership.

This semantic narrowing does **not** by itself solve physical history-owned reclamation. It defines what is allowed to own restoration so the eventual reclamation decision has a truthful causal boundary. A separate correctness defect was also exposed: a later realization to the same location must not be discarded when undoing an earlier authored write. That repair belongs with structural reversal work, not with diagnostic history.

13.4 Step 8

OPEN. History-aware reclamation must answer:

When retained history has logically stopped owning a retired subject - for example because a bounded history entry was evicted - how does the physical layer learn that the lifetime is now safe to reclaim?

Plausible directions include:

exposing time-travel restore reachability to the common eligibility assessor;

giving the assessor a second history input shape;

converging ownership representation if a broader architectural reason justifies it.

The project explicitly rejects forcing timeTravel onto TurnStore solely to solve memory if that coupling is not otherwise earned.

---

14. Causal architecture: transactions, undo, and realization

14.1 The causal kernel is not synonymous with undo

Source coupling showed a useful fracture line:

```
transactions
  -> AppliedHistory
  -> TurnStore
  -> pending rollback
  -> realization context
  -> tree realization adapter
  -> causal types

timeTravel
  -> reversal/effect types
  -> tree realization adapter
```

transactions is the deeper client of the causal runtime. timeTravel mainly needs a way to apply reversal effects back to the tree. This distinction matters for Step 8 because it explains why a TurnStore-based ownership assessor cannot automatically see every time-travel restore path.

14.2 Authorship is different from realization

The v15 model keeps these concepts separate:

```
user-authored mutation
    -> may create causal history

rollback / undo / restore
    -> changes physical truth
    -> realizes previously authored meaning
    -> should not be mistaken for a fresh user-authored mutation
```

This separation is necessary to prevent feedback loops such as history recording its own undo, persistence treating speculative writes as settled external facts, or publication order exposing partially realized state.

14.3 Pending and confirmed transaction states

The transaction subsystem models lifecycle states such as open, sealed, confirmed, and aborted. Confirmation does not necessarily mean a subject can be reclaimed immediately because a confirmed turn may still be undoable. The true end of restoration ability can occur later, such as when a confirmed turn is evicted from retained history.

This is another reason that reclamation is an ownership/reachability problem rather than a simple "transaction ended" event.

14.4 Earlier time-travel economics: recording got cheap before restore did

The history work that preceded the current reclamation investigation established an important asymmetry. Structural sharing made **recording** a history entry dramatically cheaper, but it did not make whole-state **restore** constant-time.

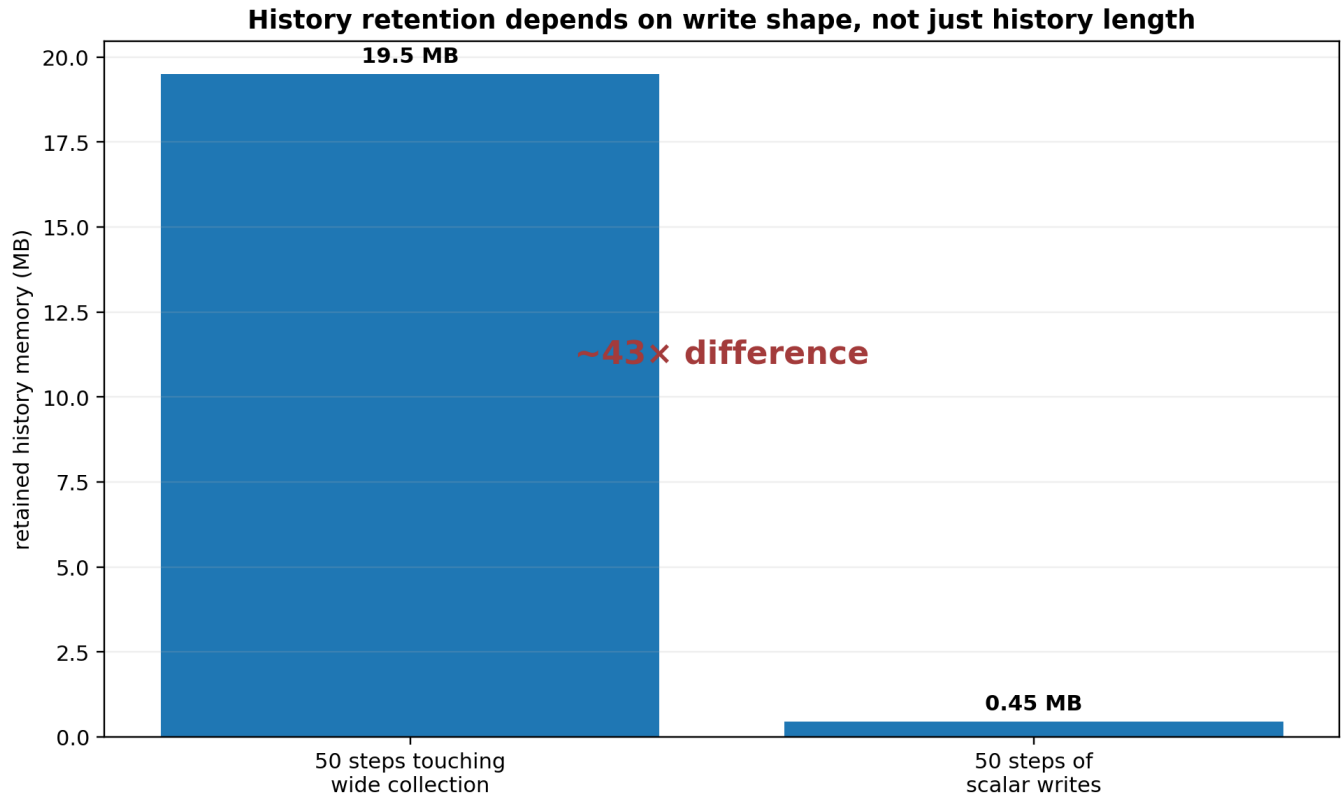
Measured earlier in the v15 program:

50 writes over a 10k-row state moved from about 340.60 ms of recording work to sub-millisecond after structural sharing improvements;

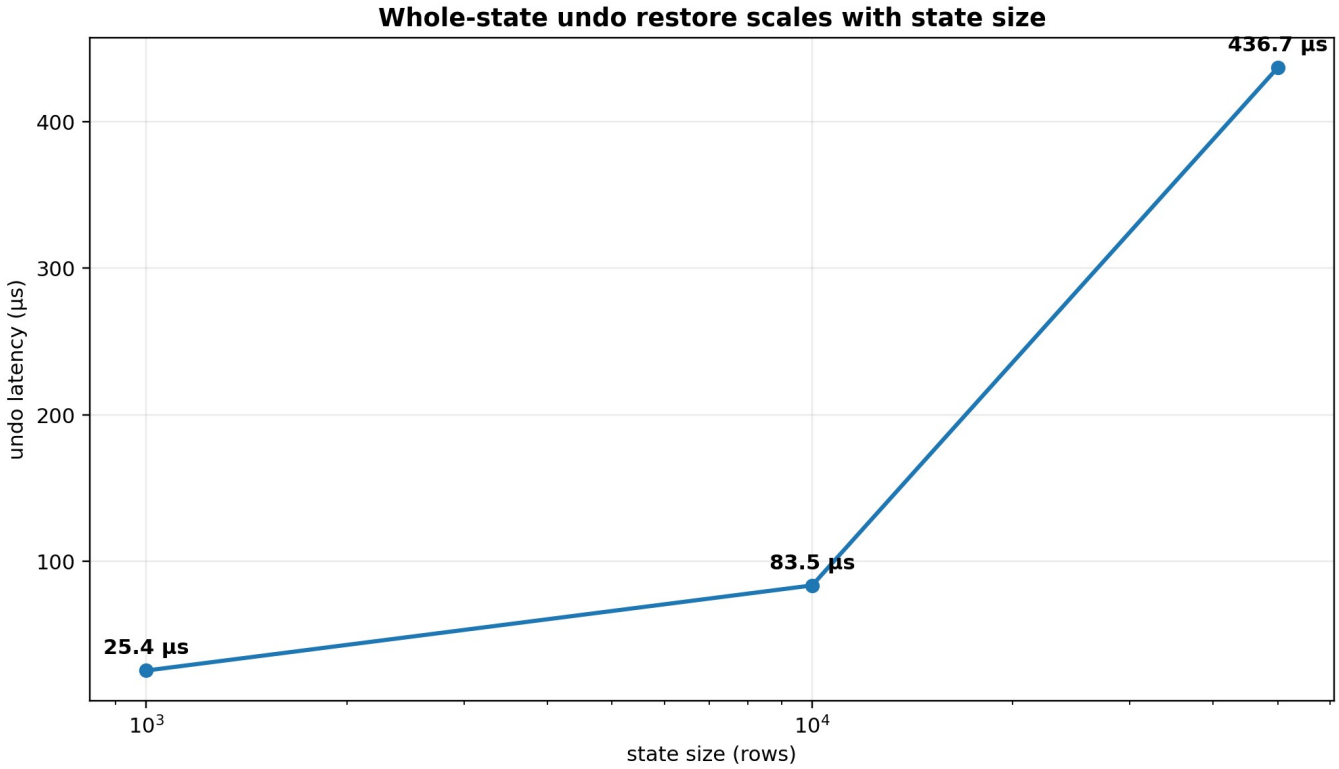
undo() through the whole-state restore path scaled about 25.4 -> 83.5 -> 436.7 us at 1k / 10k / 50k rows;

at 50 history steps over 50k rows, writes touching the wide collection retained about 19.5 MB, while scalar writes beside that collection retained about 0.45 MB - a roughly 43x difference caused by write shape;

history entry count itself grew with state shape even when retained bytes remained flatter, which is why retention limits must be expressed in semantic turns/history ownership rather than raw low-level entry counts. [E28]



What it shows: the same 50-step history can retain radically different memory depending on what each write touches. **Comparison:** wide-collection writes retained ~19.5 MB versus ~0.45 MB for scalar writes beside that collection - about a 43x difference. **Takeaway:** history limits and reclamation policy must reason about semantic ownership/write shape, not only a count of low-level entries.



What it shows: whole-state restore remains proportional to state size in the measured path. **Comparison:** undo grew from ~25.4 us at 1k to ~436.7 us at 50k even after recording became cheap. **Takeaway:** v15 does not pretend pointer-swap rewind is its unique advantage; the stronger opportunity is effect/ownership-aware history.

The conclusion was deliberately modest:

Temporal rewind is production-permissible at these measured costs, but literal whole-state rewind is not SignalTree's unique architectural advantage. The stronger opportunity is semantic, effect/ownership-aware history that knows what changed and what must remain restorable.

This is the context for the later causal-realization work and Step 8. The goal is not simply to make snapshots smaller. It is to align history retention with semantic ownership so that a subject is retained exactly while some history path can still restore it.

14.5 History participation exposed semantic asymmetry

An earlier audit also found that not every state feature participated symmetrically in time travel. In one measured state of the code, form() writes were not themselves recorded as history events, yet form values were present in later snapshots; an unrelated recorded write could therefore capture the then-current form value and a later undo could rewind it.

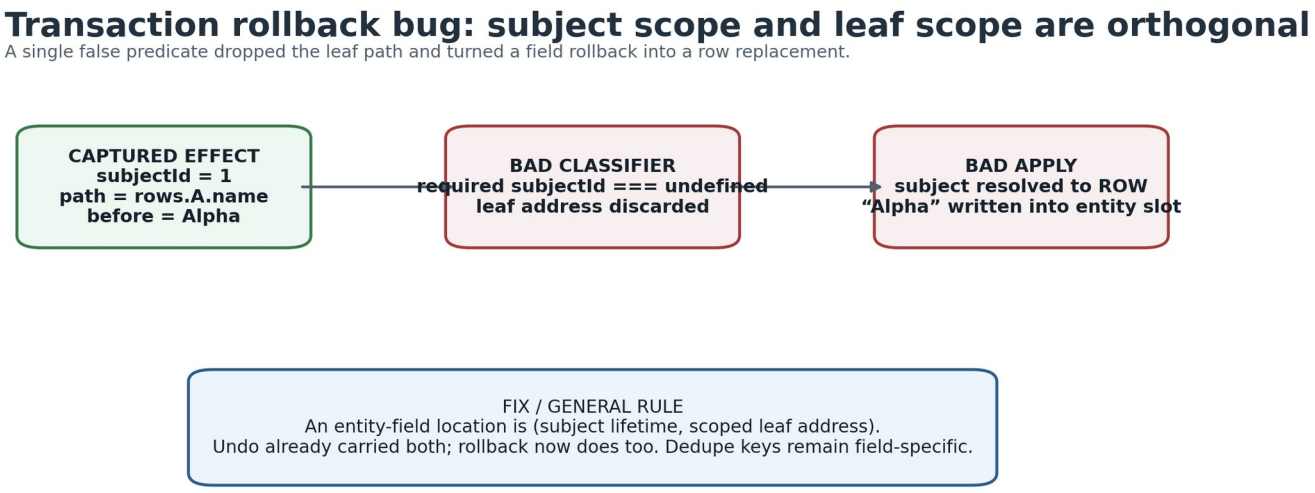
That finding was important even apart from its eventual implementation disposition because it established a release rule:

A feature cannot be excluded from history authorship but accidentally included in restoration merely because a whole snapshot happens to contain it.

The causal model must define participation intentionally. The whitepaper does not assert that the earlier form asymmetry remains on the current release line; it records the audit result because it helped motivate the separation of semantic mutation, causal authorship, realization, and publication.

---

15. Transaction rollback defect uncovered by lifecycle testing



What it shows: the rollback defect was an identity-scope bug, not a generic transaction-order bug. **Comparison:** the captured effect had both subject and leaf information; a bad classifier discarded the leaf and wrote the scalar into the row slot. **Takeaway:** semantic location for an entity field is (subject lifetime, scoped leaf address).

15.1 The failure

While adding transaction controls for the lifetime-ledger experiment, a supported public path could not be made correct:

```
transaction(() => rows.updateOne(id, patch)).rollback();
```

Instead of restoring a field on the row, rollback could replace the entire entity object with the field's previous scalar value:

```
before      [{ id: 'A', name: 'Alpha'   }]
during tx   [{ id: 'A', name: 'Changed' }]
after       ['Alpha']
```

Structural membership stayed correct. ids() remained right and all() still had the expected length, so a list could render the correct number of items while field reads silently became undefined. This was data corruption hidden behind structurally correct collection shape.

15.2 Root cause

The captured effect was already correct and contained both dimensions:

```
{
  "subjectId": 1,
  "path": "rows.A.name",
  "ownerPath": "rows",
  "before": "Alpha",
  "after": "Changed"
}
```

A classifier named hasInlineScopedLeafAddress required effect.subjectId === undefined. That incorrectly treated subject identity and leaf address as alternatives. The effect was routed to a branch that dropped its path. The applier then had a subject ID but no field path, resolved the whole row, and wrote the scalar value into it.

The undo planner had always carried both dimensions, which is why undo was correct while transaction rollback was not.

15.3 A second bug shared the same cause

Rollback deduped effects by a key. Once the scoped leaf path had been discarded, two field writes on the same row collapsed to the same owner key. Only the first field was restored; later fields could remain at transactional values.

The same fix restored field-specific addressing and therefore fixed both defects.

15.4 Architectural rule

This bug turned an implementation detail into a clear semantic model:

For an entity field, semantic location is (subject lifetime, scoped leaf address). Subject and leaf are orthogonal dimensions.

The corrected spec contains direct entity-field rollback rows plus controls for confirm behavior and scalar-leaf rollback. The fix was kept in its own commit rather than folded into retention work so both conclusions remained independently falsifiable.

---

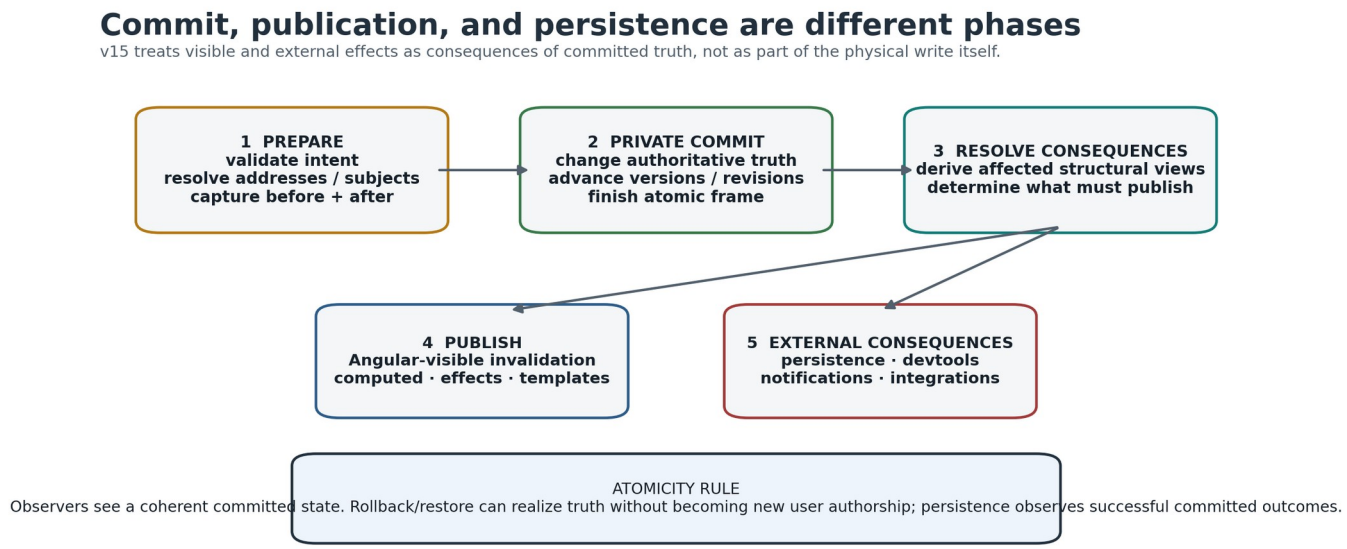
16. Reactive publication, persistence, and post-commit consequences

16.1 Commit first, publish after

The broader v15 kernel work converged on a phase discipline: physical truth should become coherent before outward consequences observe it.

A useful conceptual sequence is:

```
PREPARE
  -> determine intended physical/semantic changes

COMMIT AUTHORITATIVE TRUTH
  -> update coherent physical state

PUBLISH
  -> invalidate/update Angular-visible reactive surfaces

CONSEQUENCES
  -> persistence, devtools, external notification, other post-commit observers
```



What it shows: physical commit, reactive publication, and persistence occur at different semantic phases. **Comparison:** restore/rollback may change truth without creating new authorship; persistence should observe a successful committed outcome rather than participate in the write itself. **Takeaway:** external systems never need to see a half-applied multi-position frame.

Older design sketches sometimes called an intermediate stage "PROJECT." After the permanent materialized entity projection was deleted, that word should not be read as requiring an always-maintained entity snapshot cache.

16.2 Persistence is a consequence, not mutation authority

One of the v15 direction changes was to avoid persistence observing speculative transactional writes and then needing compensating persistence on rollback. The desired semantic boundary is closer to:

```
speculative transaction writes
        |
  transaction settles
        |
committed semantic outcome
        |
  persistence consequence
```

The core insight is independent of adapter details: persistence should observe settled committed truth; it should not become a hidden participant in causal authorship.

16.3 Framework observation is also a consequence of truth

Angular signals/computeds/effects are the observation surface through which application consumers see state changes. They are not the semantic owner of entity lifetime or causal history. The weak-interner experiment is a particularly sharp example: an Angular dependency edge did not imply the lifetime of a weakly interned SignalTree subject signal.

That is why the architecture avoids treating "Angular dependency exists" as a substitute for SignalTree's own lifetime semantics.

16.4 link() is a relationship, not a generic side-effect hook

The production Link API was selected only after a series of differential and falsification probes around directionality, settlement, echo suppression, errors, entity collections, and ownership. Its job is to model one

relationship between an owned SignalTree source and an endpoint that may provide acquisition (get/subscribe) and/or egress (set).

Inbound endpoint values are applied through SignalTree's external-acquisition path rather than authored as ordinary local writes. Outbound values are scheduled as durable post-commit consequences so rolled-back speculative work does not escape. Egress is serialized, and a later authored value remains pending until the endpoint has acknowledged an equivalent current value.

The architectural boundary is:

SignalTree owns causal settlement.
Link exposes relationship settlement.
Application code decides what to do with settled state.

This is why v15 does not publish a generic afterCommit()/onCommitted() primitive merely to implement Link. The public relationship sits on private settlement machinery.

16.5 The Link boundary is full-value even when the tree is granular

COMPARISON-FULL-STATE-0 closed the remaining semantic question: Link exchanges **complete values**, not patches. A scalar endpoint receives a scalar, a branch endpoint receives the complete branch value, and an entity collection crosses the boundary as the complete Row[] snapshot from all()/setAll(). Inbound [B, C] over [A, B] means replacement with [B, C], not merge-preserving A.

Production makes one equality decision:

```
if (knownY !== undefined && deepEqual(now, knownY.value)) return;
```

That one rule serves echo suppression and acknowledgement/reconciliation. The current deepEqual semantics include SameValueZero primitives, arrays, plain objects, dates, regular expressions, maps, sets, errors, boxed primitives, and bounded cycle handling; functions remain equal only by reference. v15 does not expose a custom comparator because no earned behavior required one.

The phrase **full-state** therefore describes the **Link boundary**, not SignalTree's internal mutation granularity. Entity notifications, reversal, position identity, and reactive invalidation remain fine-grained internally.

One mutation deliberately remains unresolved rather than hidden: removing the reconciliation loop's explicit continuation currently kills no production test because notifier-driven flush rescheduling performs the follow-up send. The loop is retained as belt-and-braces; deleting it would require its own falsifier proving flush-driven rearm across lifecycle, disposal, and settlement cases.

16.6 Link failures are observable without becoming Link state

A rejected outbound endpoint set() produces one public onTreeError event with operation: 'link:set', the owning TreeId, and the linked state path. The authored source value remains in SignalTree, the outbound queue remains usable, and settled() resolves rather than turning transient endpoint failure into handle state. A throwing error listener is isolated from Link and from peer listeners.

This contract deliberately keeps Link at exactly three methods. Failure observation belongs to the generic process-wide channel; reporting internals remain private.

---

17. Tree lifecycle and destroy()

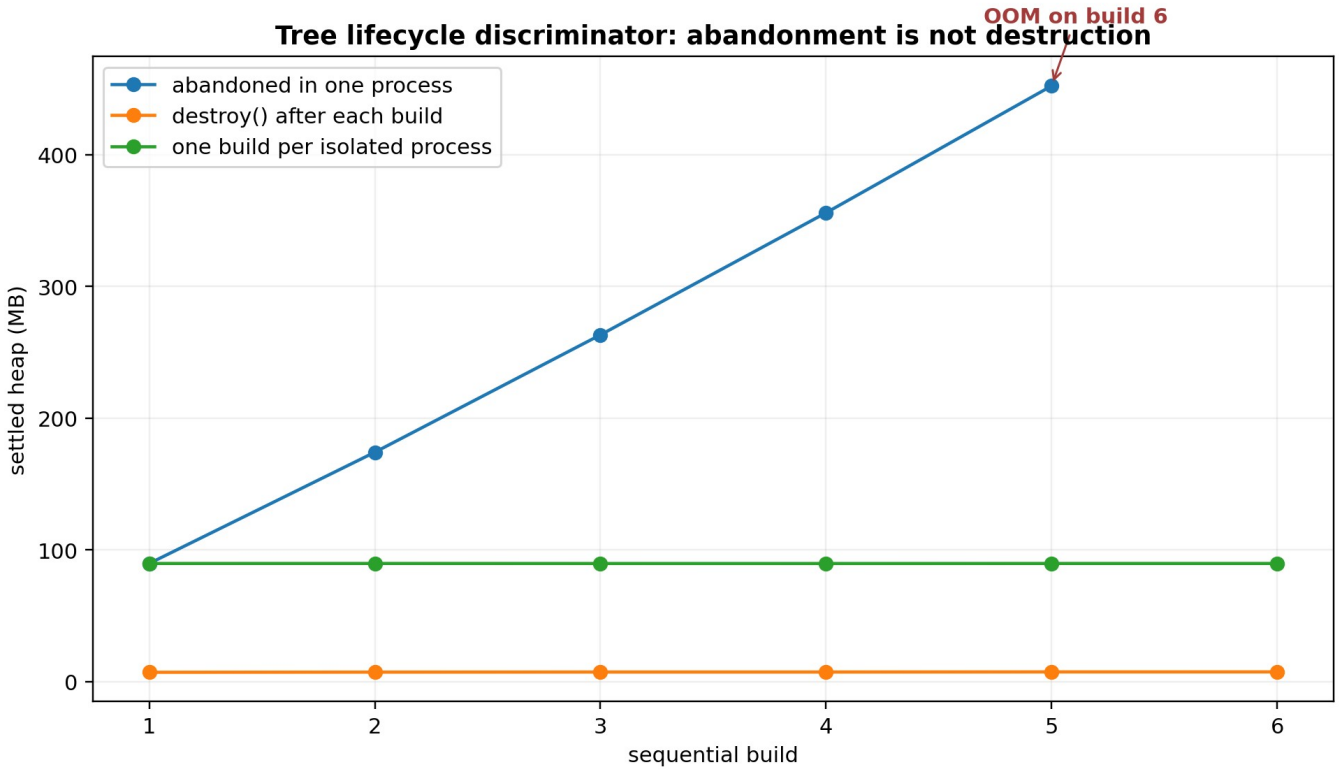
Subject lifetime and tree lifetime are separate state machines.

TREE
create -> active -> destroy()

SUBJECT
live -> retire -> {restorable | zero-owner} -> reclaim/forget

17.1 The discriminator

A late v15 memory investigation measured repeated store creation in three controlled modes:



What it shows: repeated abandoned stores accumulate, repeated destroyed stores stay flat, and one store per isolated process is individually stable. **Comparison:** the in-process growth was ownership accumulation, not a write-history leak inside one tree. **Takeaway:** destroy() is an API/lifecycle contract that must be documented for tests, SSR, routes, and temporary stores.

	build 1	build 2	build 3	build 4	build 5	build 6
abandoned	89.65	174.08	263.14	355.81	452.13	OOM
destroyed	7.08	7.21	7.28	7.28	7.37	7.38
isolated	89.65	89.66	89.63	89.62	89.65	89.65

The isolated processes are stable to two decimals, so one store does not exhibit per-update or per-build internal growth in this discriminator. The in-process abandoned curve is accumulation of whole store lifetimes. Adding destroy() between builds collapses that accumulation.

The probe returned nothing from its per-build helper, excluding the theory that the benchmark result closure itself held each store.

17.2 The corrected interpretation

The finding is:

A write-active SignalTree remains retained until destroy() is called. This is a lifecycle contract, not evidence that the store itself grows without bound while live.

One build cost about 84.3 MB at zero wide updates and about 94.99 MB at 400 wide updates in the same investigation. History contributed only about 27 KB per wide update; the approximately 89 MB retained signature was overwhelmingly seeded store/runtime state, not a history-recording loop.

An earlier setup-only script had suggested sublinear accumulation because it used heapUsed after a single gc() rather than the repository's quiescence protocol. That old conclusion was corrected in place rather than erased.

17.3 Why this matters beyond benchmarks

Long-lived applications may create only a few global stores. Other environments create bounded-lifetime trees repeatedly:

tests;

SSR/request handlers;

route-scoped state;

temporary editing sessions or tools;

dynamically mounted application regions.

Those consumers need explicit lifecycle guidance. The core README/agent guidance did not yet state this requirement at the time of the discriminator, so documenting it belongs in release hardening.

17.4 The quarantined OOM cell is a different problem

Adding destroy() did **not** resolve the one outstanding featured / update-100-fields @ 10k matrix cell. Inside that harness, seeding scales around 142 KB/row:

n=1000 151.8 MB
n=2000 292.2 MB
n=5000 714.8 MB
n=10000 1418.1 MB

Standalone reproductions - including the harness's own SignalTree implementation code executed inside the harness process - settle near roughly 8 KB/row / 82 MB. The anomaly is linear and reproducible but unlocalized.

Three hypotheses were tested and rejected:

cross-tree time-travel contamination through global PathNotifier state;

retention of the seed array;

superlinear construction of a third tree.

The correct disposition is **QUARANTINED**. That benchmark row must not be quoted as a SignalTree result until the harness discrepancy is explained.

---

18. Current entity performance and memory baseline

18.1 Public 10k collection-layer benchmark

A current 10k public-layer run records approximately:

operation

median

plain array construct

1.60 ms

entityMap declaration

0.72 ms

setAll

18.16 ms

addMany

18.14 ms

updateOne

0.19 ms

updateOne + dependent read

0.33 ms

addOne

0.19 ms

removeOne

0.31 ms

changeId

0.24 ms

projection all()

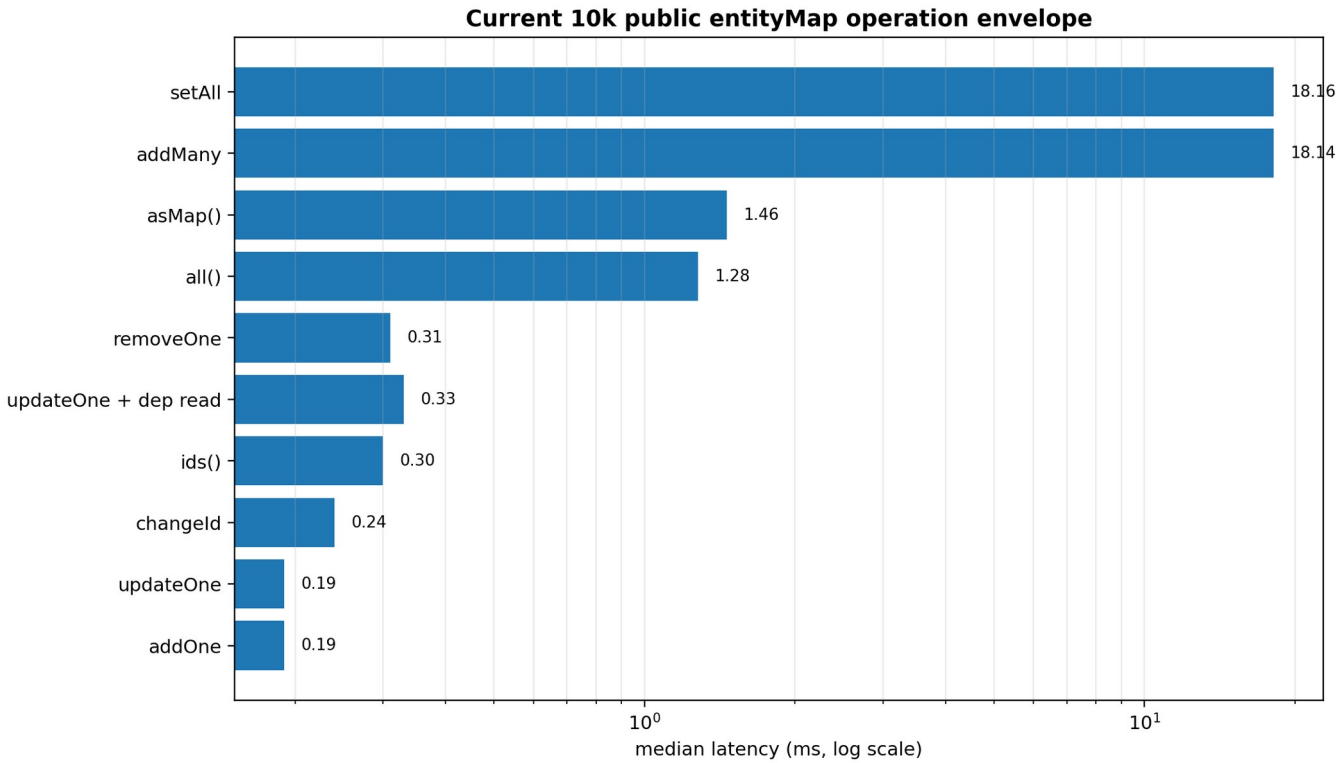
1.28 ms

projection ids()

0.30 ms

projection asMap()

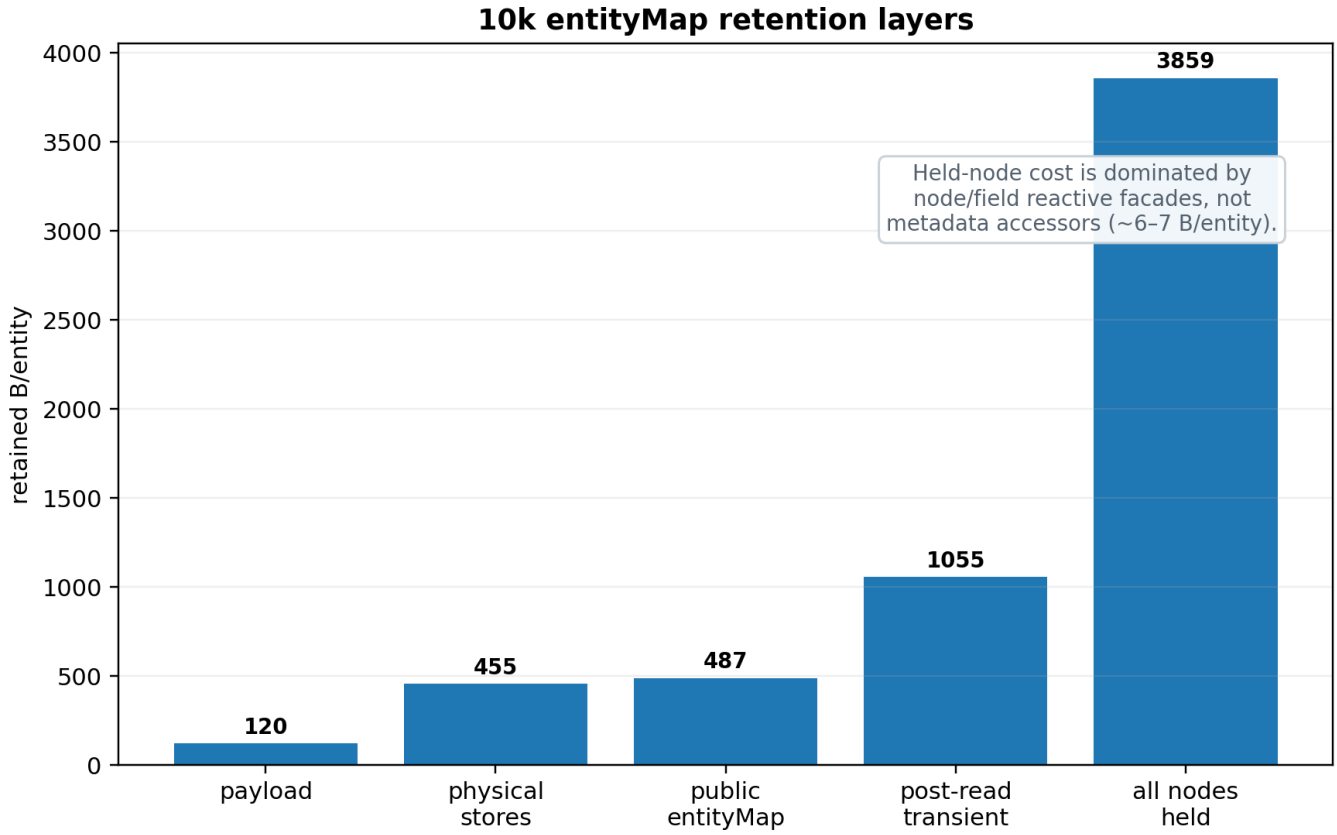
1.46 ms



What it shows: current 10k public-layer latency spans roughly two orders of magnitude between bulk population and local point operations. **Comparison:** setAll/addMany are ~18 ms, whole projections are ~1.3-1.5 ms, and local update/add/remove/rekey operations are ~0.2-0.3 ms in this harness. **Takeaway:** the operation class matters; one headline latency cannot describe the entity engine.

These figures belong to this public-layer harness. They should not be mixed directly with sub-microsecond per-operation loop harnesses without labeling the different measurement context.

18.2 Live retention layers



What it shows: live baseline storage is far cheaper than holding every row/node/field facade. **Comparison:** public entityMap baseline is ~487 B/entity; holding all node/field views is ~3,859 B/entity, while metadata accessors

change only ~6-7 B/entity. **Takeaway:** the old hypothesis that property metadata represented a ~1.7 KB/entity prize was disproved.

At 10k entities with three fields each and the quiescence protocol:

layer

retained

B/entity

interpretation

payload floor

1.14 MB

120

data only

physical stores

4.34 MB

455

authoritative entity storage

entity semantics, no metadata

4.59 MB

481

includes Angular realization

entity semantics

4.59 MB

481

metadata flags before node realization

public entityMap baseline

4.64 MB

487

normal live collection baseline

post-read transient residue

10.06 MB

1,055

after full read, nodes dropped

every row/node/field held

36.80 MB

3,859

dominant held-facade cost

held nodes, metadata disabled

36.75 MB

3,853

control

The last pair corrected another important earlier claim. Metadata accessors account for only about **6-7 B/entity**, not a previously reported ~1,710 B/entity. Most held-node cost is node/field reactive realization itself.

This is why the property-vs-sidecar representation trial is deferred: its potential memory prize is now small compared with solved/unresolved lifecycle costs.

18.3 Workload-class benchmark

Assumption-based workloads are reported with construction and steady-state time separately:

workload

N

construction

steady state

B/entity

POINT_HEAVY

10k

18.23 +/- 7.57 ms

45.18 +/- 37.38 ms

468

PROJECTION_HEAVY

10k

14.49 +/- 2.80

477.44 +/- 26.58

464

REACTIVE_FANOUT

10k

14.80 +/- 14.31

221.24 +/- 13.88

465

BULK_LOAD

100k

159.32 +/- 24.88

283.88 +/- 44.02

406

REALTIME

10k

13.73 +/- 3.45

850.50 +/- 252.08

466

The +/- is max-min spread across samples, not a statistical confidence interval. A delta inside the spread is treated as inconclusive.

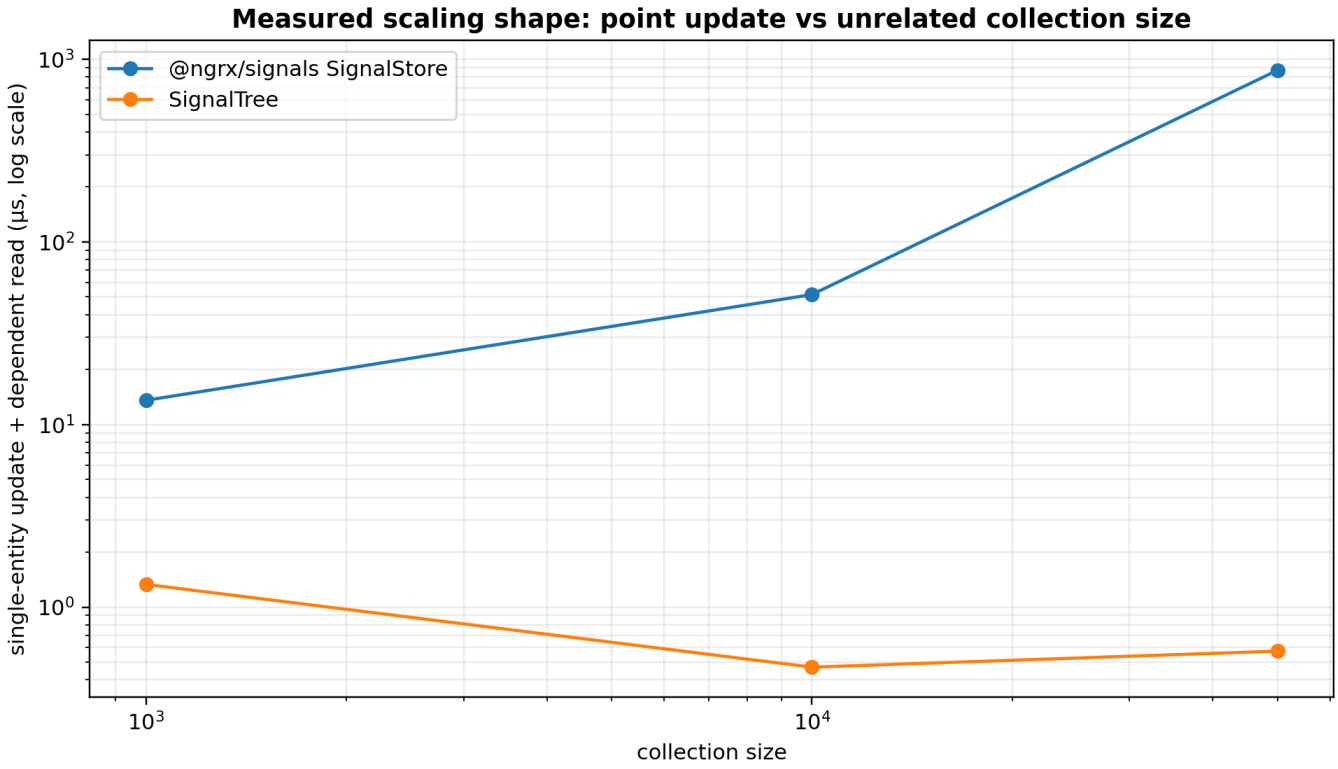
Construction and steady state stay separate because a read-path change cannot causally improve construction; summing the two lets construction variance masquerade as a runtime win.

---

19. Point-update scaling and competitor context

This section deliberately uses three different comparison views because they answer different questions. The scaling chart asks whether cost grows with unrelated collection width. The task chart compares like-for-like fixture latency on several operations. The small-N cross-library chart asks whether SignalTree pays extra fixed semantic overhead when the collection is too small for scaling behavior to dominate. No single chart is allowed to stand in for all three questions.

19.1 The architectural claim is a shape



What it shows: SignalTree point-update cost stays effectively flat as unrelated collection size grows in this fixture; the compared SignalStore path grows sharply. **Comparison:** the useful result is the different slope, not a single multiplier at one N. **Takeaway:** the evidence supports "not inherently O(n) in unrelated collection width" for this point-update path, not a blanket claim that every update is O(1).

A current interleaved comparison against @ngrx/signals 21.1.1 measured single-entity update plus dependent read as collection size grew:

collection size

SignalStore

SignalTree

1,000

13.57 us

1.331 us

10,000

51.47 us

0.469 us

50,000

870.45 us

0.574 us

The useful claim is not one dramatic multiplier. The multiplier changes with N. The architectural result is the scaling shape: on this path, SignalTree remains effectively flat with respect to unrelated collection size while the compared SignalStore path scales with collection size because it rebuilds the entity map. [E10]

A more careful complexity description is:

```
point-update cost =
    O(address resolution)
  + O(local mutation bookkeeping)
  + O(affected reactive work)
```

With n unrelated entities and k genuinely affected consumers, the important result is "not inherently O(n)." It is not a claim that arbitrary fan-out is strict O(1).

19.2 Other measured tasks

The same current comparison reported approximately:

task

SignalTree

SignalStore

deep field write, 10 levels

0.011 us

1.109 us

update 1 row of 50k + dependent read

1.075 us

795.337 us

write then whole-state read x10

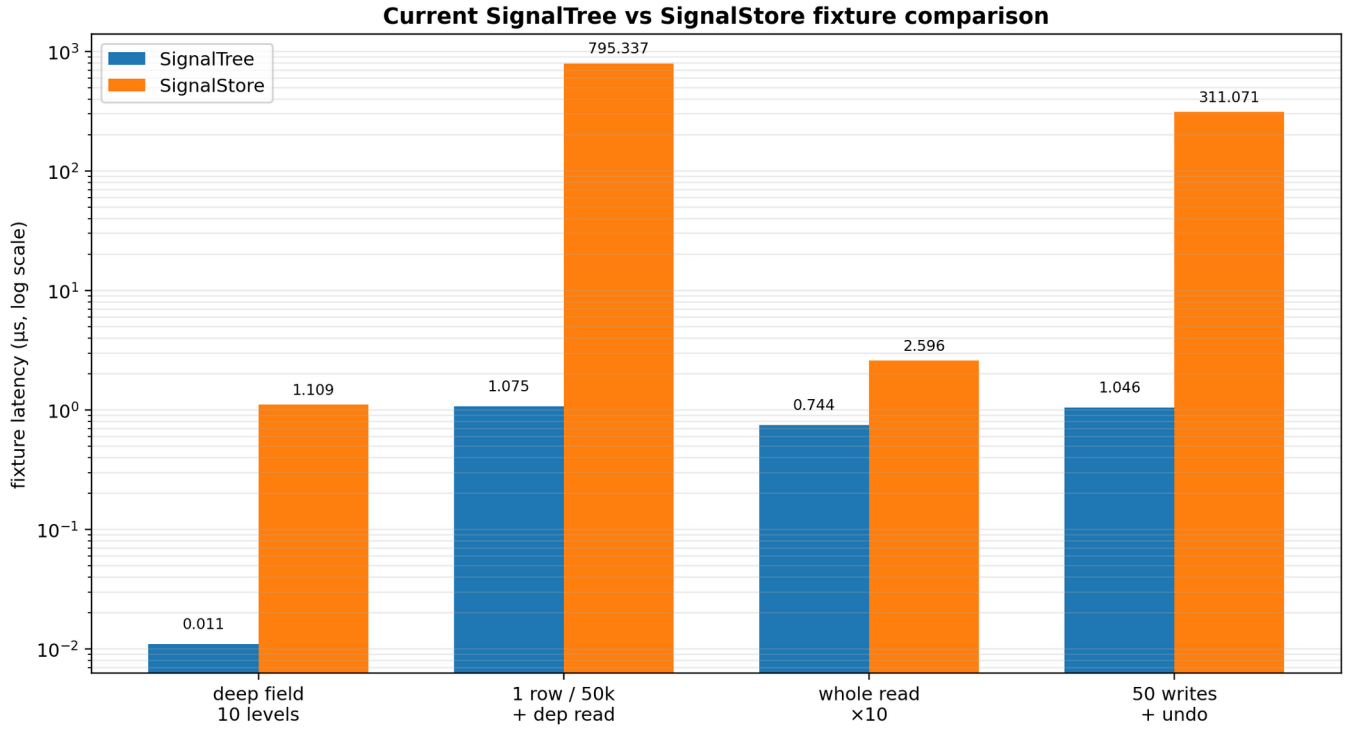
0.744 us

2.596 us

50 writes with undo

1.046 us

311.071 us



What it shows: several current fixture comparisons on one log scale. **Comparison:** deep point writes and 50k entity updates are especially favorable to SignalTree; the whole-state read difference is much smaller, and the history arm is capability-shaped. **Takeaway:** use this chart to understand workload shape, not to declare a universal library winner.

These are fixture results, not universal library rankings. The undo comparison is also capability-shaped: SignalTree has built-in history semantics, while the comparison implementation must construct equivalent behavior.

19.3 Small-N cross-library result is deliberately unflattering

At n=200, a cross-library benchmark measured:

implementation

collection arm

undo/redo arm

history mechanism

raw Angular signals

0.10 ms

5.87 ms

hand-rolled

Elf

0.26 ms

0.14 ms

built-in

NgRx Signals

0.51 ms

3.97 ms

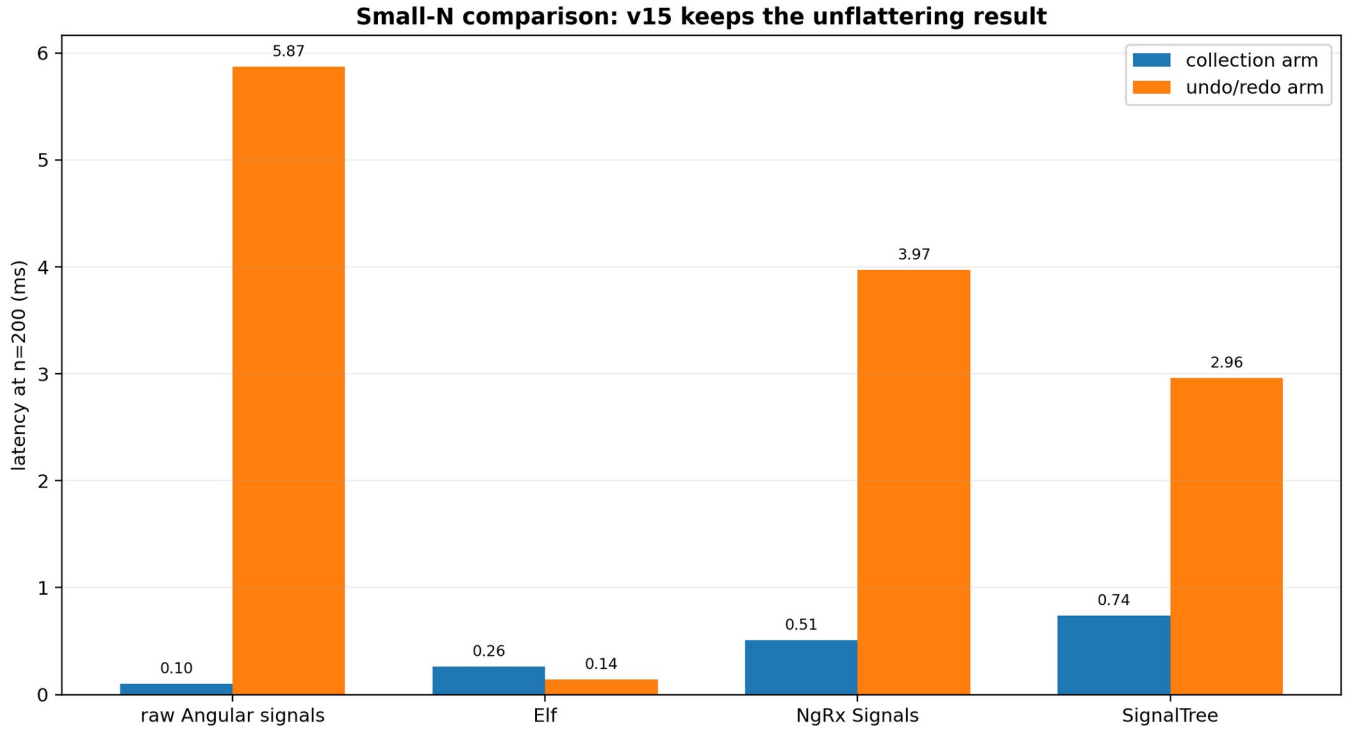
hand-rolled

SignalTree

0.74 ms

2.96 ms

built-in



What it shows: at n=200, fixed overhead dominates and SignalTree does not win the collection arm. **Comparison:** Elf wins both measured arms; raw Angular signals win the simple collection arm but pay much more in the hand-rolled history arm. **Takeaway:** v15 deliberately preserves evidence that is unfavorable when it helps separate fixed semantic cost from large-N scaling behavior.

Elf wins both arms at this size. SignalTree is last on the 200-entity collection arm. The repo intentionally keeps this result because the v15 claim is not "SignalTree wins every microbenchmark." The claim is that its architecture keeps local mutation cost from scaling with unrelated collection width while carrying richer semantics such as subject lifetime, transactions, history, and reclamation.

19.4 Raw signals are a useful floor, not a fair feature-equivalent competitor

A Map<ID, Signal<Entity>> can be extremely fast. It also omits much of SignalTree's semantic system: structural restoration, subject lifetime isolation, transactions, causal history, configuration validation, and lifecycle reclamation.

The useful question is therefore:

How close can SignalTree remain to the cost floor of raw localized reactivity while preserving the additional semantics it promises?

---

20. Step 7.5 update matrix checkpoint

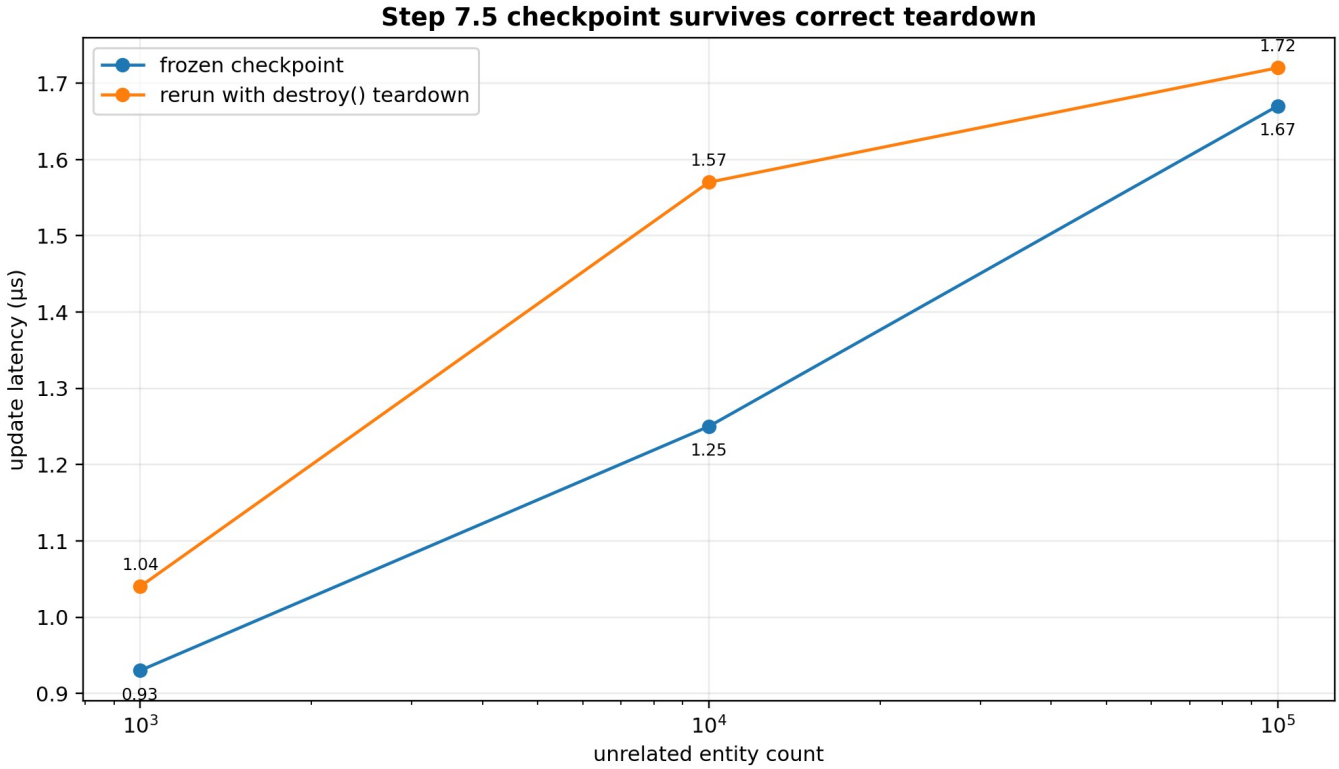
A broader update matrix was used to test whether point-update behavior remained stable across 1k, 10k, and 100k unrelated entity counts.

The checkpoint was frozen with an earlier SignalTree result around:

0.93 / 1.25 / 1.67 us

A rerun after adding correct teardown measured:

1.04 / 1.57 / 1.72 us



What it shows: the frozen and teardown-correct reruns follow the same shallow scaling shape from 1k to 100k unrelated entities. **Comparison:** absolute values moved slightly, but not enough to support a new architectural conclusion. **Takeaway:** adding correct lifecycle teardown did not invalidate the point-update checkpoint because teardown occurs outside the timed region.

The differences were ordinary variance and the scaling shape was unchanged. Because destroy() occurs outside the timed region, no re-freeze was required.

The single 100-field/10k "featured" cell that still OOMs is excluded from this checkpoint and remains quarantined as described in the lifecycle section.

21. Bundle and construction economics

Current performance documentation records approximately:

target

production gzip

budget

bare SignalTree

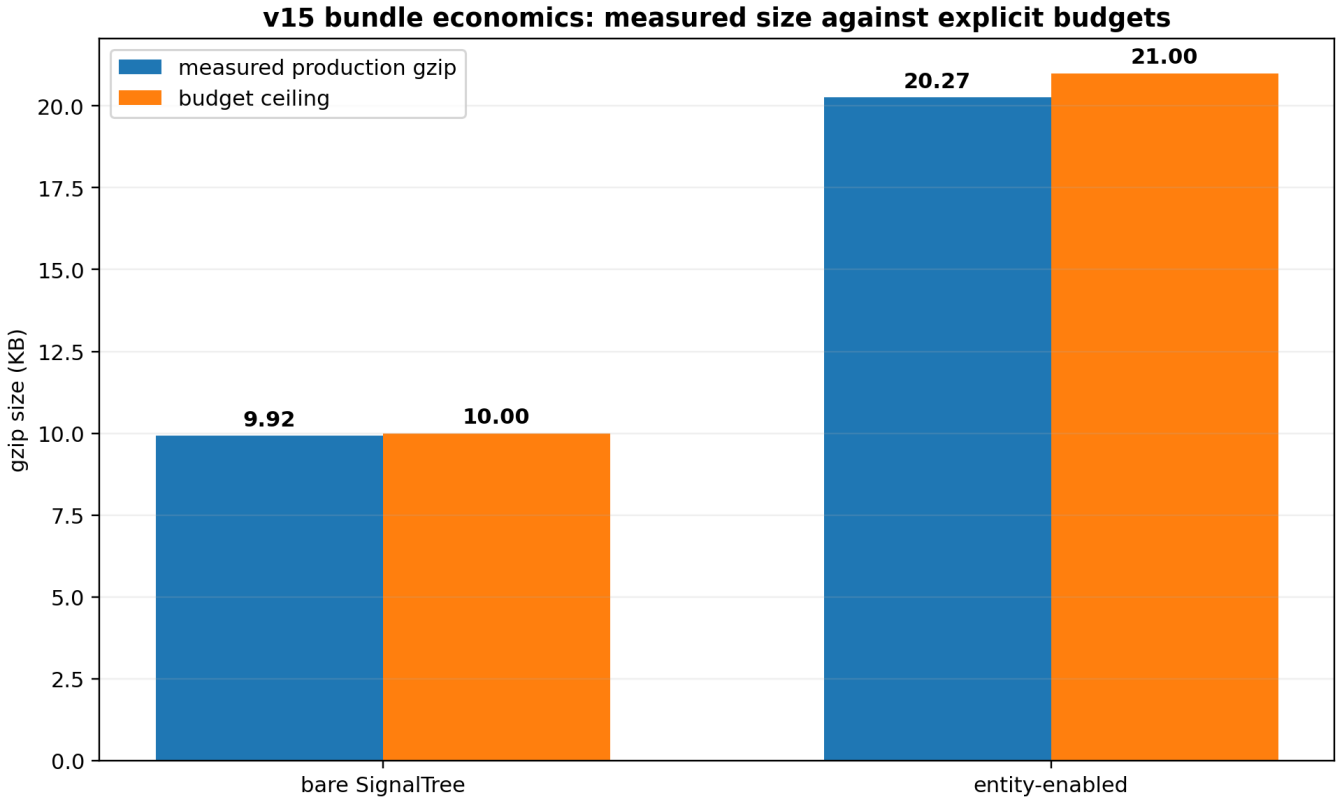
9.92 KB

10.0 KB

entity-enabled

20.27 KB

21.0 KB



What it shows: both the bare and entity-enabled builds remain inside explicit gzip ceilings. **Comparison:** the bare build sits close to its 10 KB budget; declarative planning consumed intentional budget rather than being treated as free. **Takeaway:** v15 optimizes within product budgets instead of treating minimum byte count as the architecture.

The declarative construction migration intentionally consumed roughly 0.47 KB gzip of the bare budget. This is an example of v15 treating bundle bytes as a budgeted resource rather than an absolute optimization target. The architecture would not reintroduce an ambiguous legacy constructor simply to win back that space.

Likewise, construction latency is not summed into steady-state performance when assessing read/write architecture. v15 explicitly permits one-time compilation/materialization work if it buys simpler, cheaper repeated operations.

---

22. Benchmark methodology that survived the audit

The performance work generated several durable rules.

22.1 Always rebuild and interleave A/B arms

One initial comparison suggested setAll and updateOne had regressed by roughly 21% and 50%. The two builds had been measured sequentially, one against stale output. After rebuilding and interleaving, the ranges overlapped and the new path was marginally faster in both ordering rounds. The correct conclusion was **no demonstrated regression**.

22.2 Quiescence is part of memory measurement

Memory results use a quiescence helper rather than one immediate global.gc(). Garbage collection, finalizers, and event-loop cleanup can require multiple cycles/turns. The later tree-lifecycle discriminator directly corrected an earlier wrong conclusion caused by violating this rule.

22.3 A/A controls and spread matter

A candidate delta must exceed the harness noise floor. Workload-class results inside max-min spread are recorded as inconclusive. This prevents small favorable changes from being promoted into architectural evidence.

22.4 Pre-register the criterion before the fix

The retired-subject experiment pre-registered "growth must stop," not merely "bytes should decline." That forced the project to call the first 249 -> 117 B result incomplete. Only whole-lifetime forgetting satisfied the asymptotic criterion.

22.5 Gate the gate

Self-tests prove that custom release gates actually fail under a relevant mutation. The retired-slope gate initially had a blind self-test because mutating one threshold did not bypass a second ratio condition. The self-test was corrected. At the latest architecture checkpoint, the gate suite reported 39/39 self-tests proven, zero blind, zero errored.

22.6 Quarantine unresolved cells

The 100-field/10k OOM discrepancy is not averaged away or explained by guesswork. It is excluded from quotable results until the harness/standalone disagreement is localized.

---

23. Rejected alternatives and why

23.1 Keep and use the materialized projection

Why attractive: real 25-40% uncached all() improvement at medium/large N.

Why rejected for v15: permanent ~126 B/entity; benefit concentrated in one shared reconstruction; no broader semantic use; absolute no-projection performance satisfies the release envelope.

23.2 Keep the projection because history may need it later

Rejected: speculative future reuse does not justify permanent current cost. History/sync need temporal deltas such as before/after/version/origin/transaction, not merely today's ordered snapshot.

23.3 Weakly intern entity signals

Why attractive: all ordinary tests passed and memory improved dramatically.

Rejected: forced GC showed silent stale consumers. Correctness function survived; weak representation did not.

23.4 Permanently retain retired lifetime/revision tombstones

Why attractive: appeared to support stale-handle isolation.

Rejected: hard-delete/forget null preserved stale-handle semantics and bounded zero-owner churn. Permanent ledger was unearned.

23.5 Reclaim every retired subject unconditionally

Rejected: deliberately false eligibility causes undo to throw because required backing has been deleted. Time-travel control arms prove owned retention is semantically real.

23.6 Mutable runtime restoration-owner registry

Why once considered: under .with(), restoration authority could attach after construction.

Rejected after phase-model change: the enhancer set is immutable before runtime, so zero-owner authority is a static value. A mutable container for an immutable fact would be machinery that outlived its reason.

23.7 Preserve .with() as a compatibility shim

Rejected: capability-requiring enhancers applied after materialization fundamentally cannot participate in truthful planning. A shim would preserve exactly the lifecycle ambiguity v15 is removing.

23.8 Publish both signalTree() and plannedSignalTree()

Rejected: the planned path was the right engine but the wrong product shape. v15 merged its planning model into the single public constructor and deleted the duplicate path.

23.9 Switch property metadata to sidecar during construction migration

Rejected as a combined change: it confounded two independent variables and produced unrelated structural failures. Property metadata was restored for the construction commit; sidecar remains an isolated, low-priority A/B with only ~6-7 B/entity apparent live-node prize.

23.10 Optimize the quarantined OOM cell by guessing

Rejected: three plausible hypotheses were already falsified. The row stays quarantined rather than motivating an architecture change from an unexplained harness artifact.

---

24. Corrections that materially changed the architecture

v15's credibility comes partly from the number of times the project corrected itself.

24.1 Production usage counts were wrong

A grep pipeline stripped file paths before a test filter was applied. Corrected counts reduced the entityMap declaration count and changed how whole-vs-point reads were interpreted.

24.2 Whole reads were over-counted semantically

Ten all().find(...) sites were alternate-key point lookups forced through scans. API usage was not equal to user intent.

24.3 A read-modify-write pattern was misclassified

A whole-collection replacement workaround existed because exact replacement was missing in the pinned version. It was not evidence that point writes should reconstruct collections.

24.4 CPU/memory "break-even" was a category error

No number of saved milliseconds erases retained megabytes. The project moved to a multi-dimensional architecture profile.

24.5 Weak interning looked safe until GC

The complete ordinary suite passed. A direct reachability falsifier overturned the conclusion.

24.6 Metadata accessors were not a 1.7 KB/entity opportunity

A later controlled L5/L5m comparison measured only about 6-7 B/entity. The older large number was amended and explicitly retired rather than silently disappearing.

24.7 Zero-owner reclamation was initially called a partial success

It cut 249 -> 117 B/retired but failed the pre-registered asymptotic criterion. Whole-lifetime forgetting was required to actually close Step 6.

24.8 The history loop was blamed for repeated-store memory

The lifecycle discriminator showed one build already carries most of the 89 MB signature; history growth was only tens of KB per wide update. Repeated abandoned trees accumulated because they were never destroyed.

24.9 A single-GC setup probe under-measured retained heap

That shortcut violated the repository's own quiescence rule and produced the false impression that repeated setup was sublinear. The correction is intentionally retained in the engineering record.

These corrections are not embarrassing footnotes. They are the mechanism by which the final architecture became more defensible.

---

25. Migration model from v14 to v15

25.1 Enhancer composition

Before:

```
const tree = signalTree(state)
  .with(timeTravel())
  .with(batching());
```

After:

```
const tree = signalTree(state, {
  enhancers: [timeTravel(), batching()]
});
```

Built-in enhancers are factories and are called in the array.

composeEnhancers(...) was removed as redundant; reusable enhancer bundles return arrays and are spread into the declarative set.

25.2 Conditional enhancement

Before:

```
const tree = isProd ? base : base.with(timeTravel());
```

After, build the configuration rather than mutating a live tree:

```
const tree = signalTree(state, {
  enhancers: isProd ? [] : [timeTravel()]
});
```

This is more than syntax. A production tree with no time travel now gets a truthful non-causal plan rather than the old maximal legacy plan.

25.3 Derived state

Chained .derived() remains supported under its existing finalization rules. When configuration is already being supplied, derived state can also be declared in config so the full construction shape is visible together. Enhancers are applied before derived realization.

25.4 Late enhancer attachment is intentionally impossible

The old notion "attach an owner later, but it gets no retroactive history" has been replaced by a stronger structural rule:

all authorities are configured before runtime state mutation begins

A tree cannot acquire time travel after it has already retired subjects. History cannot become retroactive because late attachment is no longer an operation.

25.5 Public-surface migration is validated from tarballs

v15 release engineering includes package builds and external-consumer typechecks rather than relying only on monorepo source imports. This matters because declaration emit, root-barrel exports, package subpaths, and actual published artifacts can fail while source-level tests stay green.

An earlier hardening snapshot treated missing stored/form root exports as a likely barrel defect. Subsequent production-surface work corrected that framing for stored: it is not part of the final root surface and is scheduled for migration/retirement rather than promotion. Package-root validation remains essential, but “missing export” is not automatically a bug; the export must first be earned as final public contract.

25.6 Legacy async and persistence primitives are now a migration phase

The final-v15 surface audit intentionally separated "superseded" from "safe to delete immediately." asyncSource retirement was measured as a broad migration touching production, tests, demo, tools, and documentation. Its one central reportTreeError call was removed first because it covered only the synchronous-throw path while the marker's supported local error() signal covered synchronous throws, Observable errors, and Promise rejections. Full primitive retirement remains a later compatibility phase.

MIGRATION-MAP-0 then corrected an important earlier classification. loader, asyncSource, stored, flushAllStoredSignals, and asyncQuery are **not reachable from the published package surface**: the package export map exposes only the root and package.json, the root barrel does not export those symbols, and the internal markers barrel is not itself published. Their retirement is therefore internal/demo/doc/tool migration rather than a new public breaking change. The earlier statement that asyncSource was publicly exported merely because markers/index.ts exported it was wrong; package reachability, not an internal barrel, is the authority.

The migration inventory also found that asyncQuery had not yet been dispositioned even though it has live production and demo usage, so it now gets a dedicated classification step before loader retirement. stored remains the heaviest semantic case: its behavior includes serialization/deserialization, storage injection, debounce/max-wait, versioning/migration, local error context, read/write/migrate/remove operations, global error participation, a global drain, and transaction interaction. The project therefore still rejects the shortcut "stored is superseded by Link." The migration question is behavioral: if these primitives had never existed, which final-v15 mechanisms would naturally express the required behavior?

25.7 The demo becomes a release-gate consumer

The demo is no longer treated as a collection of examples that merely compile. After compatibility cleanup it must be rebuilt against the final public surface and exercise the supported architecture using package-root APIs. Required scenarios include ordinary tree state, entity collections, retained enhancer capabilities, external(), Link in all three directions, explicit retrieval, strong settlement, disposal, collection snapshots, public error observation, TreeId attribution, state-path diagnostics, and failed-send recovery.

The negative gate is equally important: zero internal imports, zero deleted APIs, zero experimental Link modes, zero patch/Partial Link semantics, and no test helpers. This makes the demo an ergonomics and coverage layer between core invariants and a real production consumer.

---

26. Release validation system

26.1 The architecture checkpoint moved substantially

The earlier 1,799-test snapshot in this paper was a useful checkpoint, not the final v15 line. After Link, public error attribution, full-state comparison, and evidence consolidation, the current core suite at this cut reports:

2,192 passed / 0 failed
20 skipped / 1 todo

Lint and typecheck are green. The lower count than the immediately preceding 2,250-test run is intentional: CONSOLIDATION-0 retired 58 archaeological tests after proving their falsifiers were subsumed by production-facing invariant batteries. No production or public API code changed in that consolidation commit.

26.2 Public Link/error conformance is now a permanent gate

The production Link battery and public tree-error batteries pin:

retrieve() participates in whole-Link settlement;

held consequence, outbound queue, and retrieval work are included in settled();

disposal releases waiters and suppresses late acquisition;

same-shaped trees receive distinct TreeId values while the same tree is stable;

path is the SignalTree state location for both Link and stored;

one failed send produces one public event;

listener failure is isolated;

authored state and queue usability survive endpoint rejection;

the Link handle remains exactly retrieve / settled / dispose;

full-value collection replacement and fresh-but-structurally-equal echo suppression remain intact.

26.3 Evidence consolidation is itself part of release hardening

Three historical mode harnesses (LINK-HANDLE-0, LINK-HANDLE-1, LINK-ECHO-1) each carried local experimental Link implementations. A catastrophic production mutation that prevented outbound sends from arming failed nine production-facing spec files and **none** of those three harnesses. They were therefore archived to the engineering record and deleted from the permanent suite. Their winning outcomes remain protected by production conformance.

The ERROR-SURFACE archaeology was similarly reduced from eight files to three intent-focused batteries. Source-text assertions in that area dropped from 67 to 13, with compile-time/public/runtime proof preferred whenever possible.

26.4 Full release gates are intentionally deferred until migration, demo, and perf-proof cleanup

The old six-red-gate table is historical and should not be read as the current release queue. A full release-gate run is intentionally postponed until the compatibility surface is migrated, the demo has full final-v15 coverage, and the brittle performance proof is corrected. This prevents unrelated legacy/demo/perf noise from being confused with architecture regressions.

The current sequence is:

ASYNC-QUERY-DECIDE-0
  -> LOADER-RETIRE-0
  -> ASYNC-SOURCE-RETIRE-1

-> STORED-PERSISTENCE-0
-> STATUS-RESIDUE-0
-> MIGRATION-CLOSE-0
-> DEMO-COVERAGE-0
-> PERF-PROOF-0
-> FULL RELEASE GATES
-> Candidate B refreeze
-> TruckTrax migration

A separate local ChatGPT code-review session reported three plausible release-hygiene issues: a tarball-consumer check still expecting an unpublished @signaltree/core/storage subpath, an Angular consumer fixture calling toWritableSignal with the wrong shape, and missing core-README coverage for the newly public link/onTreeError surface. That review also described a staged/unstaged working tree and test counts inconsistent with the later committed consolidation/migration-map sequence. This paper therefore treats those findings as **ADVISORY, NOT ESTABLISHED** until they are reproduced against the current clean HEAD. They are useful hypotheses for the eventual gate pass, not current architecture claims.

26.5 The known timing flake remains quarantined from correctness claims

A wall-clock assertion in entity-granular-reactivity has been reproduced as a timing flake: it can fail in the full suite and pass repeatedly in isolation. The release plan is not to loosen the threshold opportunistically. PERF-PROOF-0 must replace fragile single-sample timing logic with warmup/repetition and a robust statistic or otherwise state the performance guarantee more truthfully.

26.6 Isolated commits remain a release rule

Architectural and hardening changes continue to be isolated so falsifiers remain attributable. The recent Link/error sequence followed this rule: production Link landing, production conformance, error-surface falsification, owner identity, async-source reporter retirement, internal error repair, public export, full-state comparison, and evidence consolidation all landed separately.

---

27. Commit milestones

The following commits are useful architectural checkpoints from the engineering record. They are not intended as a complete git history.

7896addf - remove materialized entity projection

db4dbcc5 - realize subject activation tokens on demand

896ab368 - delete unearned subject-position transport

7e573b95 - repair retention/benchmark methodology

1b10f7ff - gate reactive identity durability across GC

d5333830 - pin prospective restoration authority under the old chain model - later structurally superseded by static declarative authority

0a23a551 - capability authority audit and corrections

223b355a - breaking declarative enhancer construction; delete .with()

7959f6bc - delete obsolete planned-vs-chain A/B tool

87a790eb - zero-owner value/signal reclamation at retirement boundary

5c74381a - v15 performance baseline and retirement of old metadata-memory claim

982c378b - falsify the retired-subject lifetime ledger

91043109 - forget entire zero-owner subject lifetime

d487a4ae - fix entity-field transaction rollback corruption

c3d79be0 - land production link() relationship

c2f993a0 - production Link conformance; make retrieval participate in whole-Link settlement

d4d97b9f - ERROR-SURFACE-1 falsifies the old unattributed reporter event

e7c48f38 - prove registry identity is the correct live-tree namespace

420269d6 - scope full asyncSource retirement as migration rather than a narrow prerequisite

afe08611 - repair internal error event: required branded TreeId, delete dead source/detail

a09aef9c - close ERROR-SURFACE-2; publish onTreeError, TreeErrorEvent, TreeId; unify path semantics

a4456012 - close COMPARISON-FULL-STATE-0; complete-value Link boundary, one equality rule

89f142d5 - consolidate archaeological tests/harnesses without production or API changes

5c949a24 - complete MIGRATION-MAP-0; correct package reachability, inventory retiring primitives, insert asyncQuery disposition before deletion phases

One important lesson from this timeline is that a prior committed semantic rule can legitimately be superseded when a later architecture removes the state transition that made the rule necessary. The prospective-authority rule was correct for a world where .with() could attach after construction. Once v15 deleted that transition, static authority became the stronger and simpler model.

---

28. What SignalTree v15 is - technically

The most precise high-level model at this architecture cut is:

SignalTree v15 is a planned, fine-grained reactive state engine for structured TypeScript state. Its public state namespace is hierarchical, while its reactive dependency topology may be a graph. Configuration is finalized before runtime so optional machinery can be selected from the complete capability set. Entity and other subject-backed state can carry non-reused lifetime identity distinct from reusable keys, physical addresses, and mutation revisions. Subject-scoped reactive references remain associated with the lifetime from which they were acquired rather than silently following a later occupant of the same address. Physical mutations become coherent at explicit commit boundaries and reactive/external consequences observe committed truth. External systems can be connected through a small link() relationship whose boundary exchanges complete values, while private settlement machinery preserves causal correctness. Rejected outbound sends are observable through a generic tree-attributed error channel rather than becoming Link status. Point updates are localized to affected state and affected reactive work rather than inherently scaling with unrelated collection members. A zero-owner retired subject can be reclaimed and completely forgotten; a history-owned subject remains until causal eligibility proves restoration is impossible.

And separately:

A SignalTree object itself owns runtime resources for its lifetime. Bounded-lifetime consumers must call destroy() when that tree is no longer needed.

This definition deliberately avoids making Angular Signal object identity the conceptual center. Signals are an observation mechanism. Addresses provide navigation. Subjects provide lifetime semantics. Storage provides physical truth. Causal structures provide meaning over time.

---

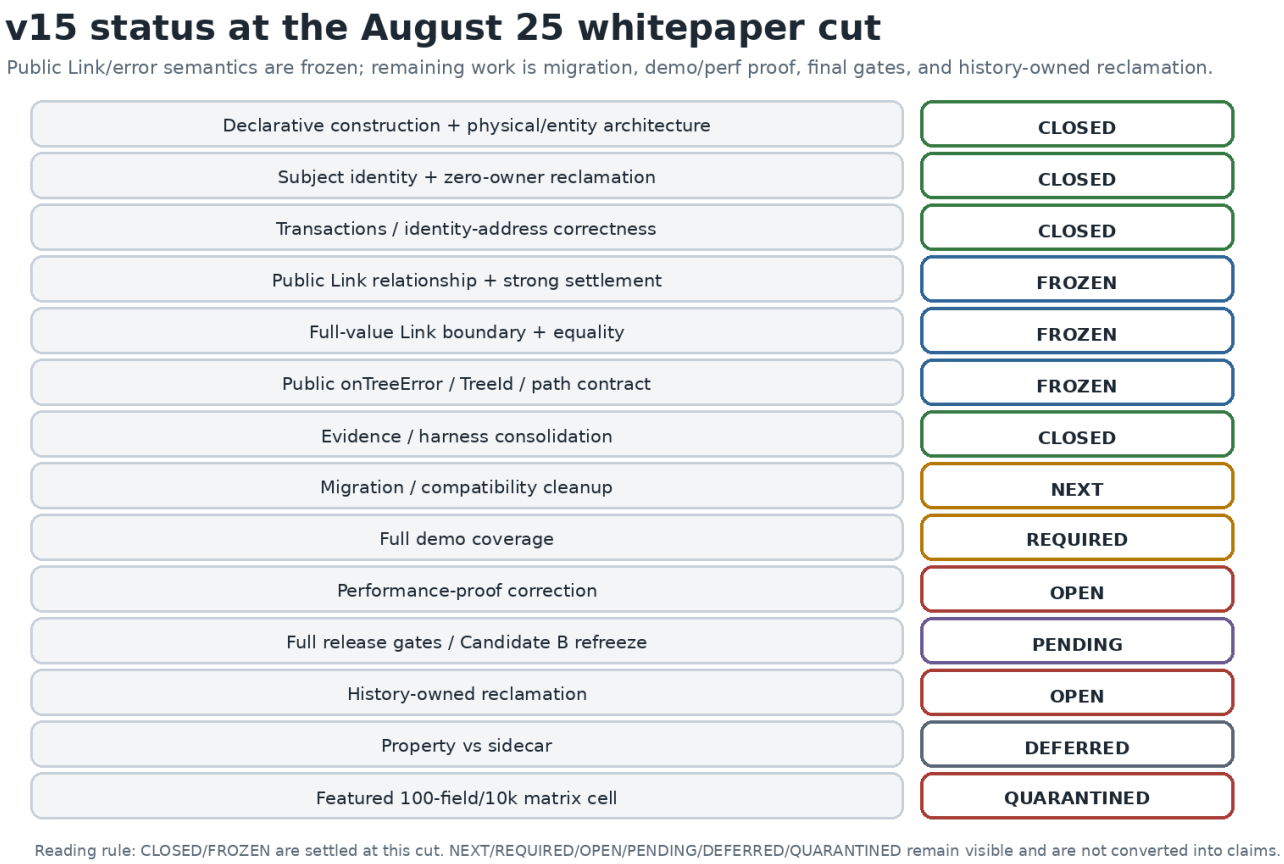
29. What v15 does not claim

The engineering record supports several explicit non-claims:

- **Not:** "SignalTree preserves signal identity forever."
  Instead: subject-scoped reactive semantics remain correct for the relevant subject lifetime; representations may change.
- **Not:** "SignalTree updates are O(1)."
  Instead: measured point-update cost is flat with respect to unrelated collection size on tested paths; affected consumer work can still scale.
- **Not:** "SignalTree is literally a tree of signals."
  Instead: the state namespace is hierarchical; dependency relationships form a graph.
- **Not:** "Subject IDs are JavaScript object identities."
  Instead: they are logical non-reused lifetime identities.
- **Not:** "Removed entities are immediately destroyed."
  Instead: history may legitimately retain a retired subject; zero-owner subjects can now be forgotten immediately.
- **Not:** "Tombstones are required for stale-handle safety."
  Instead: Step 6 disproved the need for a permanent central tombstone in the zero-owner case.
- **Not:** "Memory is O(live entities)."
  Instead: history-retained subjects still contribute until Step 8 supplies history-aware reclamation.
- **Not:** "SignalTree is always faster than Elf/NgRx/raw signals."
  Instead: individual microbenchmarks vary; the strongest current performance claim is the local-update scaling shape under the tested semantics.
- **Not:** "Every benchmark row is publishable."
  Instead: the 100-field/10k featured cell is quarantined.
- **Not:** "Link is a patch/merge protocol."
  Instead: Link exchanges complete values; collections cross as complete Row[] snapshots.
- **Not:** "Link exposes causal settlement machinery."
  Instead: the public handle exposes retrieve(), settled(), and dispose(); notifier/held-consequence machinery stays private.
- **Not:** "TreeId is a persistent or globally meaningful tree address."
  Instead: it is a runtime-local correlation namespace for distinguishing live trees in diagnostics.
- **Not:** "onTreeError observes every caught exception in the library."
  Instead: it observes errors SignalTree explicitly reports through the global channel; the measured producer set is intentionally narrow.

---

30. Current status and remaining work



What it shows: the architecture is now closed farther into the external-synchronization and diagnostics surface than the August 22 cut; the remaining queue is primarily migration, demo coverage, performance-proof repair, final gates, and the still-open history-owned reclamation problem. **Comparison:** Link/error semantics are frozen; compatibility and release proof remain deliberately separate. **Takeaway:** the project is no longer searching for more public Link API. Remaining consolidation is evidence/implementation cleanup, not surface growth.

30.1 Architecture closed or frozen

declarative construction and .with() deletion;

enhancer set validation and dependency ordering;

truthful capability planning;

permanent entity projection deletion;

activation-token laziness and subject-position cleanup;

GC-durable subject-scoped reactive identity;

zero-owner value/signal reclamation and whole-lifetime forgetting;

transaction entity-field rollback correctness;

identity/address theorem: PositionId is causal/ownership position, SubjectId is entity lifetime, field path is coordinate;

production link() and its three-method handle;

strong whole-Link settlement including retrieval;

complete-value Link boundary and structural equality;

public onTreeError / TreeErrorEvent / TreeId attribution contract;

one coherent public error path meaning;

comparison/error-surface archaeology consolidated into permanent invariant batteries;

point-update scaling checkpoint;

property metadata retained as current representation pending any future isolated trial.

30.2 Migration / compatibility - next

MIGRATION-MAP-0 is now **closed** at commit 5c949a24. It established that none of the retiring set (loader, asyncSource, stored, flushAllStoredSignals, asyncQuery) is reachable from the published package surface, correcting the earlier assumption that an internal markers barrel made asyncSource public. The work is still broad, but the risk is internal/demo/doc/tool migration rather than a new consumer-facing break.

The inventory also found 23 tools/scripts referencing retiring APIs, including release-gate machinery, so those references must move with their primitive instead of being left to fail later for migration reasons. The demo already has zero internal/deep imports and substantial Link usage, but the public error channel is barely demonstrated; full demo reconstruction remains a required gate after compatibility cleanup.

The working order is now:

ASYNC-QUERY-DECIDE-0
- -> LOADER-RETIRE-0
- -> ASYNC-SOURCE-RETIRE-1
- -> STORED-PERSISTENCE-0
- -> STATUS-RESIDUE-0
- -> MIGRATION-CLOSE-0

This is intentionally not one giant deletion. Each primitive must have a measured replacement/disposition and live consumer counts across production, tests, demo, tools, and current documentation. Historical audit references remain historical rather than being rewritten out of existence. stored in particular still requires its own behavior matrix rather than being relabeled as Link.

30.3 Demo coverage is a required release gate

After migration, the demo will be completely updated to the final v15 surface and treated as a production-like public consumer. It must cover ordinary state, entity collections, retained enhancer capabilities, external(), Link scalar/branch/collection flows, get/subscribe/set, retrieve, settled, dispose, full-value replacement, echo suppression, onTreeError, TreeId, path attribution, and failed-send recovery.

The demo must contain zero internal imports, deleted APIs, experimental Link modes, test helpers, or patch semantics. If a normal example requires casts or explicit generic rescue, that is an ergonomics finding rather than something to hide in example code.

30.4 Performance proof correction

The current wall-clock entity-granularity assertion is known flaky. It remains separate from correctness work. PERF-PROOF-0 must make the proof robust before the final release gates are run.

30.5 History-owned reclamation remains open

Zero-owner reclamation is closed. Bounded history-owned reclamation is still a real lifecycle problem: backing can be released only when no retained causal state can legally restore the subject. The history-scope audit narrowed restoration eligibility, but physical reclamation still requires a truthful ownership/reachability integration.

30.6 Property vs sidecar - deferred

The latest control says the live held-node memory difference is only about 6-7 B/entity. The experiment can be revisited if another reason appears, but it is no longer a priority memory opportunity.

30.7 Quarantined benchmark anomaly

The featured 100-field/10k OOM row remains outside claims until localized. The correct engineering response is to improve the harness or find the retained structure, not to optimize SignalTree based on an unexplained cell.

---

31. Future architectural directions enabled by v15

v15 intentionally preserves seams for future work without pre-paying their permanent cost.

31.1 Incremental derived collections

The projection experiment suggested that the next meaningful projection optimization is more likely:

mutation delta
  -> affected derived computation
  -> incremental derived result

than simply maintaining a faster snapshot of current entities. The mutation-frame boundary is a useful place to expose coherent deltas later if real workloads justify it.

31.2 Secondary indexes / alternate-key lookup

TruckTrax provided negative-space evidence for alternate-key point lookup. Before adding a generalized index API, future work should distinguish:

unique alternate keys;

non-unique keys;

normalized keys;

compound keys;

genuinely dynamic predicates.

The v15 lesson is to characterize the missing semantic capability before choosing its physical index representation.

31.3 Framework-neutral extraction

The physical/causal separation makes a future framework-neutral kernel plausible. That does not require publishing a new kernel package in v15. Internal architectural separability and public package separability are different commitments.

If future product demand justifies React/Vue/vanilla adapters, v15's ownership rule - truth separated from Angular observation - reduces the amount of redesign required.

31.4 History ownership unification or adapters

Step 8 may reveal that timeTravel and transactions should expose a common restoration-ownership view even if they continue using different internal history structures. The architecture should converge only on the minimum semantic interface that earns itself.

31.5 Adaptive materialization

The projection decision does not mean whole-read acceleration is permanently rejected. A future architecture could materialize/retain a specialized projection only after repeated use proves it useful. v15 simply refuses to charge every collection the permanent cost up front.

---

32. Engineering lessons from the v15 program

32.1 Preserve user semantics, not implementation accidents

The breaking release created room to remove .with(), duplicate constructors, dead projections, redundant aliases, and tombstone ledgers without treating every historically observable internal shape as a user promise.

32.2 A function can survive while a representation fails

The project repeatedly separated "what semantic function is required?" from "does this data structure deserve to implement it?"

Examples:

whole projection speed useful
permanent MaterializedProjection rejected

subject-stable reactive continuity required
WeakRef interning rejected

stale-handle isolation required
permanent tombstone ledger rejected

This is one of the strongest generalizable outcomes of the work.

32.3 Controls are part of the result

The unchanged time-travel churn arms made zero-owner reclamation meaningful. The isolated-process memory arm made the destroy() conclusion meaningful. The weak-vs-strong forced-GC control made the identity conclusion meaningful. The negative type rows made API cleanup safe.

A positive benchmark without a matched control often says less than it appears to.

32.4 Failed criteria should remain visible

The project kept the 117 B zero-owner result recorded as a failed asymptotic criterion even though it looked like a major improvement. It kept the old 1.7 KB metadata claim with an amendment rather than rewriting history. It kept the wrong single-GC lifecycle conclusion as a correction.

That practice makes the engineering record explain not only what v15 is, but why earlier plausible models were rejected.

32.5 Release gates should protect semantics, not only code coverage

v15 added or strengthened gates for:

forced-GC identity durability;

retired-subject asymptotic slope;

public type semantics;

package/barrel surfaces;

numeric-claim provenance;

gate self-tests;

consumer tarball typechecks;

production Link conformance and public error attribution;

complete-value/structural-equality Link boundary;

demo public-surface coverage after migration.

The principle is that every unusual architecture optimization should come with the falsifier that would catch the exact way it could become wrong.

---

33. Final architectural principles

The v15 program converged on the following rules.

Know optional semantics before building physical state. Configuration is declarative and finalized before runtime.

Materialize only capabilities that are actually required. Build support is not semantic authority.

Keep logical address, lifetime identity, revision, and physical location conceptually separate. Coincidental equality is not a contract.

Local mutations should touch local truth. Unrelated collection width should not be paid for by a point update.

Whole projections are legitimate, but permanent acceleration must earn permanent memory.

Reactive observation does not own state lifetime. Angular dependency edges are not a substitute for SignalTree lifetime semantics.

A held historical reference must never silently retarget. Stale-handle isolation is semantic; permanent tombstones are not.

Reclaim only when restoration is impossible. Zero-owner absence is statically decidable; owned history requires causal eligibility.

Commit coherent truth before publication and external consequences. Physical mutation, causal authorship, reactive publication, and persistence are separate responsibilities.

Destroy bounded-lifetime trees. Tree lifecycle ownership is independent of subject reclamation.

Measure with controls, quiescence, interleaving, and pre-registered criteria. The harness is part of the system under test.

Delete unearned machinery. Preserve the architectural seam, not every historical implementation attached to it.

External synchronization is a relationship, not mutation authority. Link observes/acquires committed truth and exposes relationship settlement without publishing the scheduler.

A Link boundary can be full-value while internal state stays granular. Do not confuse synchronization payload shape with reactive/mutation granularity.

Diagnostics need namespace plus location. TreeId identifies the live tree; path identifies the state location; neither substitutes for PositionId/SubjectId.

Experiments should disappear once they stop protecting production. Preserve the winning invariant and the historical reason, not a local mode harness that cannot fail when production breaks.

These principles are the real v15 deliverable. Individual data structures can continue to improve inside 15.x as long as these contracts remain intact.

Appendix A. Glossary

address / key / path - Public logical navigation to current state. Reusable.

subject - One continuous logical lifetime of entity/subject-backed state.

SubjectId - Non-reused internal identity for a subject lifetime.

revision - Mutation generation/version within one subject; not a new subject.

PositionId - Semantic causal/topological identity used by realization/history machinery.

SlotIndex - Physical storage address in the kernel model; not a semantic identity.

reactive view - Angular-compatible observation surface over addressed/subject-scoped state.

stale handle - Previously acquired subject-scoped reference after its subject retires. It must not follow a later key occupant.

build capability - Optional physical machinery required by configured features.

restoration authority - Semantic ability of configured history/transaction features to restore retired state.

reclamation - Releasing physical backing that no legal future operation requires.

forgetting - Removing the central lifetime/revision record for a terminal zero-owner subject.

materialization - Construction-time conversion from declared markers/configuration into runtime structures/facades.

mutation frame - Staging/commit boundary for coherent multi-fact physical mutation.

realization - Applying a semantic restore/reversal/transaction plan to physical truth.

publication - Making committed change visible to reactive observers.

consequence - Post-commit external behavior such as persistence/devtools/notification.

Link - Public external relationship between an owned SignalTree source and endpoint capabilities (get, set, subscribe).

full-value Link boundary - Endpoint payloads are complete NaturalValue snapshots rather than patches; entity collections cross as complete Row[].

TreeId - Opaque runtime-local identity of one live tree namespace, used for correlation/attribution rather than persistence or restoration.

tree error path - SignalTree state location associated with a report; location, not identity.

quiescence - Memory-measurement protocol that allows GC/finalizer/event-loop cleanup to settle before reading retained heap.

Appendix B. Evidence register

The evidence IDs below point to repository artifacts or captured engineering runs, not to the excluded PDF.

E01 - production TruckTrax grep/classification, collection-access-profile.md: 7 entityMaps, 44 all(), 28 byId(), alternate-key and exact-replacement reinterpretation

E02 - setall-regression.md; commits db4dbcc5, 896ab368, 7e573b95: removal of eager position/activation cost; 90 ms -> ~17-20 ms setAll(10k) class

E03 - decision-no-materialized-projection.md; commit 7896addf: projection B/C trade and deletion

E04 - bench-workload-classes.mjs + workload assumptions: point/projection/fan-out/bulk/realtime synthetic workload classes

E05 - entity-signal-retention.md; weak-interning trial: weak representation memory gains and ordinary-suite pass

E06 - check-signal-identity-durability.mjs; commit 1b10f7ff: forced-GC stale-consumer failure and durable gate

E07 - capability-authority-audit.md; 0a23a551: existing capability graph, legacy all-on plan, planned path audit

E08 - phase-model-blast-radius.md: six spec files / eleven semantic tests affected by phase-model break

E09 - 223b355a: declarative construction, .with() deletion, tuple-based enhancer typing

E10 - bench-vs-signalstore.mjs, current interleaved run: point-update scaling shape and current NgRx comparison

E11 - bench-compare.mjs --n 200: small-N cross-library result including Elf win

E12 - bench-entity-layers.mjs: live entity retention layers; 6-7 B/entity metadata control

E13 - retired-subject-churn.md: pre-reclaim linear churn and 249 -> 117 B first-stage result

E14 - 87a790eb: automatic zero-owner value/signal reclamation

E15 - lifetime-ledger falsifier 982c378b: null that permanent tombstone ledger is unnecessary

E16 - 91043109; check-retired-subject-slope.mjs: whole-subject zero-owner forgetting and bounded asymptote

E17 - d487a4ae; transactions-entity-field-rollback.spec.ts: entity-field rollback and multi-field dedupe defect/fix

E18 - v15-performance-baseline.md: current public operation/memory/bundle baseline and methodology warning

E19 - latest repeated-build discriminator supplied during v15 audit: destroy() lifecycle contract and abandoned/destroyed/isolated memory curves

E20 - Step 7.5 update-matrix checkpoint: frozen point-update scale result and teardown re-run

E21 - current featured/update-100-fields OOM investigation: quarantined 10k row; three falsified hypotheses

E22 - signal-tree-type-matrix.typing.spec.ts and characterization commits: public TypeScript semantic contract and SignalTreeBase redundancy

E23 - external consumer/tarball typecheck probes: published declaration/consumer-path validation

E24 - tools/verify-gates.mjs --self-test: custom gate mutation proof; latest 39/39 self-tests proven

E25 - current full gate run: architecture checkpoint 1799 runtime tests green; six baseline-red release-hardening gates

E26 - persistent/immutable data-model survey, design-thesis-and-benchmarking-rules.md: bounded-fanout/width findings and rejected equality shortcut

E27 - slot/token atomic-state prototype: feasibility of SignalTree-owned truth with Angular observation tokens and atomic frames

E28 - history/time-travel performance audit: recording vs restore asymmetry, wide-write retention, turn-based retention reasoning

E29 - commits c3d79be0, c2f993a0; production Link conformance specs: final Link handle, retrieve-in-settlement, disposal and relationship-idle semantics

E30 - commits d4d97b9f, e7c48f38, afe08611, a09aef9c: error-surface falsification, live-tree namespace identity, required TreeId, coherent path, public observer

E31 - COMPARISON-FULL-STATE-0; commit a4456012: complete-value boundary, structural equality, collection replacement, no comparator/patch mode, surviving reconciliation-loop mutation

E32 - CONSOLIDATION-0; commit 89f142d5: archival of non-production mode harnesses, ERROR-SURFACE test consolidation, source-text assertion reduction with zero production/API diff

E33 - latest production-surface audit / migration plan: asyncSource reporter retirement, compatibility sequencing, mandatory demo coverage before final gates

E34 - MIGRATION-MAP-0; commit 5c949a24: package-reachability correction, consumer counts, asyncQuery disposition requirement, tool/gate migration inventory

E35 - local ChatGPT code-review transcript supplied Aug 25: **advisory only**: possible stale tarball subpath check, Angular consumer fixture error, and README coverage gaps; not promoted to established until reproduced on current clean HEAD

Appendix C. Reproduction and measurement tool index

tools/bench-public-collection-layers.mjs - public entity operation timings and allocation deltas

tools/bench-entity-layers.mjs - memory decomposition from payload through held row/field facades

tools/bench-workload-classes.mjs - assumption-driven lifetime workload classes

tools/bench-vs-signalstore.mjs - interleaved SignalTree vs NgRx Signals scaling/tasks

tools/bench-compare.mjs - small cross-library comparison including raw signals and Elf

tools/bench-entity-churn-retention.mjs - retired-subject churn with/without history and reads

tools/check-signal-identity-durability.mjs - forced-GC reactive identity correctness

tools/check-retired-subject-slope.mjs - asymptotic zero-owner retirement regression guard

tools/lib/heap-quiescence.mjs - settled retained-memory protocol

tools/verify-gates.mjs - release/architecture gate orchestrator

tools/check-numeric-claims.mjs - measured-number provenance ratchet

tools/api-inventory.mjs - public surface inventory/baseline checking

type matrix specs - exact TypeScript consumer contract and negative controls

consumer tarball probes - verify built/published declarations from outside monorepo source paths

production-link-conformance-0.spec.ts - strong Link settlement, retrieval, disposal, failure and handle-shape conformance

tree-error-attribution.spec.ts - TreeId uniqueness/stability and state-path attribution

tree-error-public-contract.spec.ts - package-root observer delivery, runtime event shape, listener isolation, failed-send behavior

comparison-full-state-0.spec.ts - complete-value boundary, collection replacement and structural-equality semantics

Obsolete tools/bench-position-topology-ab.mjs was intentionally deleted after the planned path became the only constructor architecture. Its transferable A/A/noise methodology survived elsewhere; the executable comparison itself no longer had two valid arms.

Appendix D. Completeness audit and gap closure

After drafting this whitepaper, the content was audited against a coverage checklist derived from the v15 engineering program. The purpose of this appendix is not to claim "nothing can ever be missing"; it is to make the scope review explicit and to distinguish filled gaps from genuinely open product work.

product definition - yes: final technical definition and explicit non-claims added

public recursive DX - yes: root/branch/leaf model and Rule 0d covered

TypeScript contract - yes: type matrix, enhancer accumulation, SignalTreeBase cleanup covered

construction/finalization - yes: old .with() failure and new declarative pipeline diagrammed

enhancer requirements/order - yes: whole-set validation and provider ordering covered

capability dependency graph - yes: graph and build-vs-authority distinction covered

physical architecture - yes: ownership model, IDs, mutation frame, publication boundary covered

early kernel north-star vs shipped reality - yes: explicitly labeled to avoid overclaiming typed-array/slot prototypes

persistent data-model survey - yes: bounded-fanout, width-vs-depth, and rejected equality shortcut added

early history economics - yes: recording/restore/retention measurements and semantic participation audit added

real application workload evidence - yes: TruckTrax counts and semantic reclassification covered

missing exact-replace capability - yes: v14 workaround and replaceOne distinction covered

alternate-key lookup gap - yes: ten disguised point scans and future index taxonomy covered

setAll regression - yes: cause class, cleanup sequence, before/after economics covered

materialized projection decision - yes: A/B/C fork, measured trade, deletion rationale, release envelope covered

reactive subject identity - yes: strong/none/weak trials and forced-GC failure covered

stale-handle/key-reuse semantics - yes: subject lifetime model and same-subject undo distinction covered

retired-subject churn - yes: pre-fix linear growth and decomposition covered

zero-owner value reclamation - yes: static authority prerequisite, controls, results covered

permanent ledger falsifier - yes: whole-lifetime forgetting, resurrection bug, slope gate covered

owned-history retention - yes: explicitly open; current 1.3-1.4 KB/retired and assessor mismatch covered

transactions - yes: causal role and entity-field rollback defect/fix covered

undo vs transaction coupling - yes: import/capability fracture described without claiming forced unification

persistence/consequence boundary - yes: semantic direction plus production link() relationship covered and separated from mutation authority

tree-level destroy() lifecycle - yes: latest discriminator and docs implication added

quarantined OOM cell - yes: isolated, numbers and rejected hypotheses included; excluded from claims

performance baseline - yes: public ops, workload classes, scaling, memory, bundles included

competitor context - yes: NgRx scaling + small-N Elf/raw signals with caveats included

benchmark methodology - yes: interleaving, quiescence, pre-registration, gate self-tests, quarantine covered

property vs sidecar - yes: latest 6-7 B/entity control and deferral covered

migration from v14 - yes: enhancer arrays, conditional config, derived, late-attachment removal, legacy-marker retirement sequencing covered

Link/external synchronization - yes: public handle, strong settlement, full-value boundary, structural equality and failure behavior covered

global error observation - yes: onTreeError, TreeErrorEvent, runtime-local TreeId, coherent state path, private reporter boundary covered

evidence consolidation - yes: local mode harnesses archived after production-mutation discriminator; permanent invariants retained

demo coverage - yes: now an explicit mandatory release gate after compatibility cleanup

release gates - yes: earlier 1799 checkpoint retained historically; current 2192-green post-consolidation checkpoint and deferred final-gate sequence covered

release sequencing - yes: migration -> demo coverage -> perf-proof repair -> full gates -> Candidate B -> TruckTrax; history-owned reclamation remains separate/open

rejected alternatives - yes: major forks and reasons consolidated

methodological corrections - yes: nine corrections recorded rather than hidden

glossary - yes: principal identities and lifecycle terms defined

evidence/reproduction index - yes: commits, docs, and tools listed

D.1 Gaps found during the audit and filled in this version

The first draft outline under-covered several areas. They were added before this artifact was finalized:

Tree lifecycle vs subject lifecycle. The latest destroy() discriminator is now a first-class section rather than a memory footnote.

Public type semantics. The type matrix, Rule 0d, SignalTreeBase falsifier, and cast-exposed DevTools defects were added so the paper is not only an entity-performance document.

Transaction correctness. The entity-field rollback corruption and the subject+leaf semantic-location rule were added as an architectural result, not merely a bug note.

Harness failures. The whitepaper now documents the single-GC error, stale-build/interleaving error, weak-GC blind spot, and quarantined OOM cell so benchmark methodology is part of the system explanation.

North-star vs shipped distinction. Early slot/token/typed-array ideas are explicitly labeled as guiding architecture rather than asserted as current v15 implementation details.

Bundle economics. Declarative construction's +0.47 KB cost and current bundle budgets were added to make construction choices economically complete.

Open owned-history reclamation. The paper explicitly stops at the zero-owner boundary and does not imply Step 8 is solved.

Migration semantics. .with() deletion is explained as an architectural necessity, with conditional enhancement and inferred additions shown in the new API.

Non-claims. A dedicated section prevents shorthand such as "all updates are O(1)" or "stable signals forever" from becoming accidental product promises.

External synchronization. The production Link surface, strong settlement, full-value boundary, equality rule, and rejected mode/comparator/patch alternatives are now first-class architecture rather than post-paper implementation notes.

Tree-attributed diagnostics. The public onTreeError contract, TreeId, coherent state path, and deletion of dead event fields are documented as an attribution theorem rather than an error-handling convenience.

Evidence consolidation. Historical mode harnesses and “resolved finding” tests are distinguished from permanent production invariants.

Demo as release proof. Full public-surface demo coverage is now an explicit pre-RC gate, not optional example maintenance.

D.2 Genuine open gaps that this paper cannot fill by prose

These remain engineering work, not documentation omissions:

classify asyncQuery, then retire/migrate loader, asyncSource, stored/persistence, status, and other superseded compatibility surfaces in isolated phases according to the completed MIGRATION-MAP-0;

rebuild the demo for comprehensive final-v15 coverage using package-root APIs only;

correct the known flaky wall-clock performance proof with a robust measurement protocol;

run the full release-gate suite only after migration/demo/perf cleanup, then refreeze Candidate B;

migrate/validate TruckTrax against the final package as the production-consumer audit;

implement and falsify history-aware reclamation for owned/time-travel lifetimes;

fix the authored-write-then-later-realization undo defect identified during history-scope work;

decide whether property-vs-sidecar is worth any future experiment after the 6-7 B/entity control;

localize or retire the quarantined 100-field/10k OOM cell;

re-run final published-package performance/memory evidence after RC packaging.

Those items are intentionally visible. A whitepaper about v15 should not convert unfinished migration, release proof, or history reclamation into architecture claims.

Closing note

The most important outcome of the v15 effort is not that SignalTree found one perfect representation. It is that the project now has a repeatable way to decide when a representation deserves to exist.

A projection that is faster can still be deleted. A weak reference that saves memory can still be rejected. A semantic guarantee that appears to require permanent metadata can still survive after that metadata is removed. A benchmark can be quarantined. A committed design rule can be superseded when the transition it governed disappears.

That is the architecture v15 is actually building: not a pile of optimizations, but a state engine whose costs, identities, lifetimes, causal rights, external relationships, diagnostic namespaces, and public contracts can be reasoned about separately - and falsified independently.